

Squirrel

Generated by Doxygen 1.9.8

1 File Index	1
1.1 File List	1
2 File Documentation	3
2.1 squirrel3_squirrel_sqbaselib_v1.cpp File Reference	3
2.1.1 Macro Definition Documentation	7
2.1.1.1 STRING_TOFUNCZ	7
2.1.1.2 TABLE_TO_ARRAY_FUNC	8
2.1.2 Function Documentation	9
2.1.2.1 __getcallstackinfos()	9
2.1.2.2 __map_array()	10
2.1.2.3 _closure_acall()	11
2.1.2.4 _hsort()	12
2.1.2.5 _hsort_sift_down()	13
2.1.2.6 _sort_compare()	15
2.1.2.7 array_append()	16
2.1.2.8 array_apply()	16
2.1.2.9 array_extend()	17
2.1.2.10 array_filter()	17
2.1.2.11 array_find()	18
2.1.2.12 array_insert()	19
2.1.2.13 array_map()	19
2.1.2.14 array_pop()	20
2.1.2.15 array_reduce()	20
2.1.2.16 array_remove()	21
2.1.2.17 array_resize()	21
2.1.2.18 array_reverse()	22
2.1.2.19 array_slice()	22
2.1.2.20 array_sort()	23
2.1.2.21 array_top()	24
2.1.2.22 base_array()	24
2.1.2.23 base_assert()	25
2.1.2.24 base_callee()	26
2.1.2.25 base_collectgarbage()	26
2.1.2.26 base_compilestring()	26
2.1.2.27 base_dummy()	27
2.1.2.28 base_enableddebuginfo()	27
2.1.2.29 base_error()	28
2.1.2.30 base_getconsttable()	28
2.1.2.31 base_getroottable()	29
2.1.2.32 base_getstackinfos()	29
2.1.2.33 base_newthread()	30

2.1.2.34 <code>base_print()</code>	30
2.1.2.35 <code>base_resurectureachable()</code>	31
2.1.2.36 <code>base_setconsttable()</code>	31
2.1.2.37 <code>base_setdebughook()</code>	32
2.1.2.38 <code>base_seterrorhandler()</code>	32
2.1.2.39 <code>base_setroottable()</code>	33
2.1.2.40 <code>base_suspend()</code>	33
2.1.2.41 <code>base_type()</code>	34
2.1.2.42 <code>class_getattributes()</code>	34
2.1.2.43 <code>class_getbase()</code>	35
2.1.2.44 <code>class_instance()</code>	35
2.1.2.45 <code>class_newmember()</code>	35
2.1.2.46 <code>class_rawnewmember()</code>	36
2.1.2.47 <code>class_setattributes()</code>	36
2.1.2.48 <code>closure_acall()</code>	37
2.1.2.49 <code>closure_bindenv()</code>	38
2.1.2.50 <code>closure_call()</code>	38
2.1.2.51 <code>closure_getinfos()</code>	38
2.1.2.52 <code>closure_getroot()</code>	39
2.1.2.53 <code>closure_pacall()</code>	40
2.1.2.54 <code>closure_pcall()</code>	41
2.1.2.55 <code>closure_setroot()</code>	41
2.1.2.56 <code>container_rawexists()</code>	41
2.1.2.57 <code>container_rawget()</code>	42
2.1.2.58 <code>container_rawset()</code>	42
2.1.2.59 <code>default_delegate_len()</code>	43
2.1.2.60 <code>default_delegate_tofloat()</code>	43
2.1.2.61 <code>default_delegate_tointeger()</code>	44
2.1.2.62 <code>default_delegate_tostring()</code>	45
2.1.2.63 <code>generator_getstatus()</code>	46
2.1.2.64 <code>get_slice_params()</code>	46
2.1.2.65 <code>instance_getclass()</code>	47
2.1.2.66 <code>number_delegate_tochar()</code>	48
2.1.2.67 <code>obj_clear()</code>	48
2.1.2.68 <code>obj_delegate_weakref()</code>	48
2.1.2.69 <code>sq_base_register()</code>	49
2.1.2.70 <code>str2num()</code>	49
2.1.2.71 <code>string_find()</code>	51
2.1.2.72 <code>string_slice()</code>	51
2.1.2.73 <code>table_filter()</code>	52
2.1.2.74 <code>table_getdelegate()</code>	53
2.1.2.75 <code>table_map()</code>	53

2.1.2.76 table_rawdelete()	54
2.1.2.77 table_setdelegate()	54
2.1.2.78 thread_call()	55
2.1.2.79 thread_getstackinfos()	55
2.1.2.80 thread_getstatus()	56
2.1.2.81 thread_wakeup()	57
2.1.2.82 thread_wakeupthrow()	58
2.1.2.83 weakref_ref()	58
2.1.3 Variable Documentation	59
2.1.3.1 base_funcs	59
2.2 squirrel3_squirrel_sqbaselib_v1.cpp	60

Chapter 1

File Index

1.1 File List

Here is a list of all files with brief descriptions:

squirrel3_squirrel_sqbaselib_v1.cpp	3
---	---

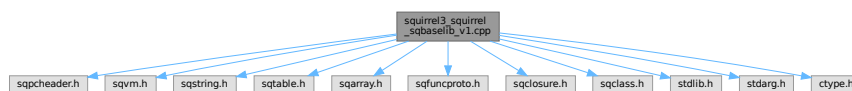
Chapter 2

File Documentation

2.1 squirrel3_squirrel_sqbaselib_v1.cpp File Reference

```
#include "sqpcheader.h"
#include "sqvm.h"
#include "sqstring.h"
#include "sqtable.h"
#include "sqarray.h"
#include "sqfuncproto.h"
#include "sqclosure.h"
#include "sqclass.h"
#include <stdlib.h>
#include <stdarg.h>
#include <ctype.h>
```

Include dependency graph for squirrel3_squirrel_sqbaselib_v1.cpp:



Macros

- #define [TABLE_TO_ARRAY_FUNC](#)(_funcname_, _valname_)
テーブルのキーまたは値を配列に変換する関数を生成するマクロ。
- #define [STRING_TOFUNCZ](#)(func)
文字列の各文字に関数を適用する関数を生成するマクロ。

Functions

- static bool [str2num](#) (const SQChar *s, SQObjectPtr &res, SQInteger base)
文字列を数値（浮動小数点数または整数）に変換します。
- static SQInteger [base_dummy](#) (HSQUIRRELVMSQ_UNUSED_ARG(v))
何も行わないダミー関数です。
- static SQInteger [base_collectgarbage](#) (HSQUIRRELVMSQ v)

- ガーベッジコレクタを実行します。
- static SQInteger [base_resurrectreachable](#) (HSQUIRRELM v)
 - 到達不能オブジェクトを復活させます。
- static SQInteger [base_getroottable](#) (HSQUIRRELM v)
 - ルートテーブルをスタックにプッシュします。
- static SQInteger [base_getconsttable](#) (HSQUIRRELM v)
 - 定数テーブルをスタックにプッシュします。
- static SQInteger [base_setroottable](#) (HSQUIRRELM v)
 - ルートテーブルを設定します。
- static SQInteger [base_setconsttable](#) (HSQUIRRELM v)
 - 定数テーブルを設定します。
- static SQInteger [base_seterrorhandler](#) (HSQUIRRELM v)
 - 新しいエラーハンドラを設定します。
- static SQInteger [base_setdebughook](#) (HSQUIRRELM v)
 - 新しいデバッグフックを設定します。
- static SQInteger [base_enableddebuginfo](#) (HSQUIRRELM v)
 - デバッグ情報の生成を有効または無効にします。
- static SQInteger [__getcallstackinfos](#) (HSQUIRRELM v, SQInteger level)
 - 指定されたレベルのコールスタック情報を取得します。
- static SQInteger [base_getstackinfos](#) (HSQUIRRELM v)
 - getstackinfos* 関数の *Squirrel API* ラッパー。
- static SQInteger [base_assert](#) (HSQUIRRELM v)
 - アサーションを評価します。
- static SQInteger [get_slice_params](#) (HSQUIRRELM v, SQInteger &sidx, SQInteger &eid, SQObjectPtr &o)
 - スライス操作のパラメータを取得します。
- static SQInteger [base_print](#) (HSQUIRRELM v)
 - 引数を標準出力に出力します。
- static SQInteger [base_error](#) (HSQUIRRELM v)
 - 引数をエラー出力します。
- static SQInteger [base_compilestring](#) (HSQUIRRELM v)
 - 文字列をコンパイルしてクロージャを生成します。
- static SQInteger [base_newthread](#) (HSQUIRRELM v)
 - 新しいスレッドを生成します。
- static SQInteger [base_suspend](#) (HSQUIRRELM v)
 - 現在のVMを中断（サスペンド）します。
- static SQInteger [base_array](#) (HSQUIRRELM v)
 - 新しい配列を生成します。
- static SQInteger [base_type](#) (HSQUIRRELM v)
 - オブジェクトの型名を取得します。
- static SQInteger [base_callee](#) (HSQUIRRELM v)
 - 現在実行中の関数の呼び出し元クロージャを取得します。
- void [sq_base_register](#) (HSQUIRRELM v)
 - ベースライブラリをVMに登録します。
- static SQInteger [default_delegate_len](#) (HSQUIRRELM v)
 - オブジェクトのサイズ（長さ）を取得するデフォルトデリゲート関数。
- static SQInteger [default_delegate_tofloat](#) (HSQUIRRELM v)
 - オブジェクトを浮動小数点数に変換するデフォルトデリゲート関数。
- static SQInteger [default_delegate_tointeger](#) (HSQUIRRELM v)
 - オブジェクトを整数に変換するデフォルトデリゲート関数。
- static SQInteger [default_delegate_tostring](#) (HSQUIRRELM v)
 - オブジェクトを文字列に変換するデフォルトデリゲート関数。

- static SQInteger [obj_delegate_weakref](#) (HSQUIRRELVm v)
オブジェクトへの弱い参照 (*weak reference*) を作成します。
- static SQInteger [obj_clear](#) (HSQUIRRELVm v)
コンテナ (テーブルや配列) の内容をクリアします。
- static SQInteger [number_delegate_tochar](#) (HSQUIRRELVm v)
数値を 1 文字の文字列に変換します。
- static SQInteger [table_rawdelete](#) (HSQUIRRELVm v)
テーブルからキーと値のペアをデリゲートを無視して削除します。
- static SQInteger [container_rawexists](#) (HSQUIRRELVm v)
コンテナ内にキーが存在するかをデリゲートを無視して確認します。
- static SQInteger [container_rawset](#) (HSQUIRRELVm v)
コンテナに値をデリゲートを無視して設定します。
- static SQInteger [container_rawget](#) (HSQUIRRELVm v)
コンテナから値をデリゲートを無視して取得します。
- static SQInteger [table_setdelegate](#) (HSQUIRRELVm v)
テーブルにデリゲートを設定します。
- static SQInteger [table_getdelegate](#) (HSQUIRRELVm v)
テーブルのデリゲートを取得します。
- static SQInteger [table_filter](#) (HSQUIRRELVm v)
テーブルの要素をフィルタリングします。
- static SQInteger [table_map](#) (HSQUIRRELVm v)
テーブルの各要素を変換 (マップ) します。
- static SQInteger [array_append](#) (HSQUIRRELVm v)
配列の末尾に要素を追加します。
- static SQInteger [array_extend](#) (HSQUIRRELVm v)
配列に別の配列の全要素を追加します。
- static SQInteger [array_reverse](#) (HSQUIRRELVm v)
配列の要素の順序を反転させます。
- static SQInteger [array_pop](#) (HSQUIRRELVm v)
配列の末尾の要素を削除し、その要素を返します。
- static SQInteger [array_top](#) (HSQUIRRELVm v)
配列の末尾の要素を削除せずに返します。
- static SQInteger [array_insert](#) (HSQUIRRELVm v)
配列の指定したインデックスに要素を挿入します。
- static SQInteger [array_remove](#) (HSQUIRRELVm v)
配列の指定したインデックスの要素を削除し、その要素を返します。
- static SQInteger [array_resize](#) (HSQUIRRELVm v)
配列のサイズを変更します。
- static SQInteger [__map_array](#) (SQArray *dest, SQArray *src, HSQUIRRELVm v)
配列の各要素に関数を適用し、結果を別の配列に格納するヘルパー関数。
- static SQInteger [array_map](#) (HSQUIRRELVm v)
配列の各要素を変換 (マップ) し、新しい配列を返します。
- static SQInteger [array_apply](#) (HSQUIRRELVm v)
配列の各要素に関数を適用し、配列をインプレースで変更します。
- static SQInteger [array_reduce](#) (HSQUIRRELVm v)
配列の要素を単一の値に集約 (リデュース) します。
- static SQInteger [array_filter](#) (HSQUIRRELVm v)
配列の要素をフィルタリングします。
- static SQInteger [array_find](#) (HSQUIRRELVm v)
配列内から指定された値を検索し、最初のインデックスを返します。

- static bool [_sort_compare](#) (HSQUIRRELVm v, SQArray *arr, SQObjectPtr &a, SQObjectPtr &b, SQInteger func, SQInteger &ret)
ソート用の比較ヘルパー関数。
- static bool [_hsort_sift_down](#) (HSQUIRRELVm v, SQArray *arr, SQInteger root, SQInteger bottom, SQInteger func)
ヒープソートのヘルパー関数。ヒープのプロパティを維持します。
- static bool [_hsort](#) (HSQUIRRELVm v, SQObjectPtr &arr, SQInteger SQ_UNUSED_ARG(l), SQInteger SQ_UNUSED_ARG(r), SQInteger func)
ヒープソートアルゴリズムの実装。
- static SQInteger [array_sort](#) (HSQUIRRELVm v)
配列をソートします。
- static SQInteger [array_slice](#) (HSQUIRRELVm v)
配列の一部を切り出した新しい配列（スライス）を返します。
- static SQInteger [string_slice](#) (HSQUIRRELVm v)
文字列の一部を切り出した新しい文字列（スライス）を返します。
- static SQInteger [string_find](#) (HSQUIRRELVm v)
文字列内から部分文字列を検索し、最初のインデックスを返します。
- static SQInteger [closure_pcall](#) (HSQUIRRELVm v)
クロージャを保護付きで呼び出します。
- static SQInteger [closure_call](#) (HSQUIRRELVm v)
クロージャを呼び出します。
- static SQInteger [_closure_acall](#) (HSQUIRRELVm v, SQBool raiseerror)
配列を引数リストとしてクロージャを呼び出すヘルパー関数。
- static SQInteger [closure_acall](#) (HSQUIRRELVm v)
配列を引数リストとしてクロージャを呼び出します。
- static SQInteger [closure_pacall](#) (HSQUIRRELVm v)
配列を引数リストとしてクロージャを保護付きで呼び出します。
- static SQInteger [closure_bindenv](#) (HSQUIRRELVm v)
クロージャに環境オブジェクトを束縛します。
- static SQInteger [closure_getroot](#) (HSQUIRRELVm v)
クロージャのルートテーブルを取得します。
- static SQInteger [closure_setroot](#) (HSQUIRRELVm v)
クロージャのルートテーブルを設定します。
- static SQInteger [closure_getinfos](#) (HSQUIRRELVm v)
クロージャに関する情報を取得します。
- static SQInteger [generator_getstatus](#) (HSQUIRRELVm v)
ジェネレータの現在の状態を取得します。
- static SQInteger [thread_call](#) (HSQUIRRELVm v)
スレッドを開始または再開します。
- static SQInteger [thread_wakeup](#) (HSQUIRRELVm v)
中断状態のスレッドを再開します。
- static SQInteger [thread_wakeupthrow](#) (HSQUIRRELVm v)
中断状態のスレッドに例外をスローして再開させます。
- static SQInteger [thread_getstatus](#) (HSQUIRRELVm v)
スレッドの現在の状態を取得します。
- static SQInteger [thread_getstackinfos](#) (HSQUIRRELVm v)
指定されたスレッドのコールスタック情報を取得します。
- static SQInteger [class_getattributes](#) (HSQUIRRELVm v)
クラスメンバの属性を取得します。
- static SQInteger [class_setattributes](#) (HSQUIRRELVm v)
クラスメンバの属性を設定します。

- static SQInteger [class_instance](#) (HSQUIRRELV v)
クラスの新しいインスタンスを生成します。
- static SQInteger [class_getbase](#) (HSQUIRRELV v)
クラスの基底クラスを取得します。
- static SQInteger [class_newmember](#) (HSQUIRRELV v)
クラスに新しいメンバを追加します。
- static SQInteger [class_rawnewmember](#) (HSQUIRRELV v)
クラスに新しいメンバをメタメソッドを無視して追加します。
- static SQInteger [instance_getclass](#) (HSQUIRRELV v)
インスタンスのクラスを取得します。
- static SQInteger [weakref_ref](#) (HSQUIRRELV v)
弱い参照 (*weak reference*) から元のオブジェクトを取得します。

Variables

- static const SQRegFunction [base_funcs](#) []
Squirrel のベースライブラリ関数を定義する静的配列。

2.1.1 Macro Definition Documentation

2.1.1.1 STRING_TOFUNCZ

```
#define STRING_TOFUNCZ(  
    func )
```

Value:

```
static SQInteger string_##func(HSQUIRRELV v) \
{ \
    SQInteger sidx, eidx; \
    SQObjectPtr str; \
    if (SQ_FAILED(get_slice_params(v, sidx, eidx, str))) return -1; \
    SQInteger slen = _string(str)->_len; \
    if (sidx < 0) sidx = slen + sidx; \
    if (eidx < 0) eidx = slen + eidx; \
    if (eidx < sidx) return sq_throwerror(v, _SC("wrong indexes")); \
    if (eidx > slen || sidx < 0) return sq_throwerror(v, _SC("slice out of range")); \
    SQInteger len = _string(str)->_len; \
    const SQChar *sthis = _stringval(str); \
    SQChar *snew = (_ss(v)->GetScratchPad(sq_rsl(len))); \
    memcpy(snew, sthis, sq_rsl(len)); \
    for (SQInteger i = sidx; i < eidx; i++) snew[i] = func(sthis[i]); \
    v->Push(SQString::Create(_ss(v), snew, len)); \
    return 1; \
}
```

文字列の各文字に関数を適用する関数を生成するマクロ。

tolower, toupper のような文字単位の変換処理を実装するために使用します。

Parameters

<i>func</i>	適用する文字変換関数 (例: tolower)。
-------------	--------------------------

Definition at line 1427 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01428 { \
01429     SQInteger sidx, eidx; \
01430     SQObjectPtr str; \
01431     if (SQ_FAILED(get_slice_params(v, sidx, eidx, str))) return -1; \
```

```

01432     SQInteger slen = _string(str)->_len; \
01433     if(sidx < 0)sidx = slen + sidx; \
01434     if(eidx < 0)eidx = slen + eidx; \
01435     if(eidx < sidx) return sq_throwerror(v,_SC("wrong indexes")); \
01436     if(eidx > slen || sidx < 0) return sq_throwerror(v,_SC("slice out of range")); \
01437     SQInteger len=_string(str)->_len; \
01438     const SQChar *sthis=_stringval(str); \
01439     SQChar *snew=(_ss(v)->GetScratchPad(sq_rsl(len))); \
01440     memcpy(snew,sthis,sq_rsl(len)); \
01441     for(SQInteger i=sidx;i<eidx;i++) snew[i] = func(sthis[i]); \
01442     v->Push(SQString::Create(_ss(v),snew,len)); \
01443     return 1; \
01444 }

```

2.1.1.2 TABLE_TO_ARRAY_FUNC

```

#define TABLE_TO_ARRAY_FUNC(
    _funcname_,
    _valname_ )

```

Value:

```

static SQInteger _funcname_(HSQUIRRELVN v) \
{ \
    SQObject &o = stack_get(v, 1); \
    SQTable *t = _table(o); \
    SQObjectPtr itr, key, val; \
    SQObjectPtr _null; \
    SQInteger nitr, n = 0; \
    SQInteger nitems = t->CountUsed(); \
    SQArray *a = SQArray::Create(_ss(v), nitems); \
    a->Resize(nitems, _null); \
    if (nitems) { \
        while ((nitr = t->Next(false, itr, key, val)) != -1) { \
            itr = (SQInteger)nitr; \
            a->Set(n, _valname_); \
            n++; \
        } \
    } \
    v->Push(a); \
    return 1; \
}

```

テーブルのキーまたは値を配列に変換する関数を生成するマクロ。

Parameters

<code>_funcname_</code>	生成される関数の名前。
<code>_valname_</code>	配列に格納する値 (key または val)。

Definition at line 820 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

00821 { \
00822     SQObject &o = stack_get(v, 1); \
00823     SQTable *t = _table(o); \
00824     SQObjectPtr itr, key, val; \
00825     SQObjectPtr _null; \
00826     SQInteger nitr, n = 0; \
00827     SQInteger nitems = t->CountUsed(); \
00828     SQArray *a = SQArray::Create(_ss(v), nitems); \
00829     a->Resize(nitems, _null); \
00830     if (nitems) { \
00831         while ((nitr = t->Next(false, itr, key, val)) != -1) { \
00832             itr = (SQInteger)nitr; \
00833             a->Set(n, _valname_); \
00834             n++; \
00835         } \
00836     } \
00837     v->Push(a); \
00838     return 1; \
00839 }

```

2.1.2 Function Documentation

2.1.2.1 __getcallstackinfos()

```
static SQInteger __getcallstackinfos (
    HSQUIRRELVM v,
    SQInteger level ) [static]
```

指定されたレベルのコールスタック情報を取得します。

指定されたスタックレベルの情報を収集し、関数名、ソースファイル名、行番号、およびローカル変数を含むテーブルをスタックに生成してプッシュします。

Parameters

<i>v</i>	Squirrel VM のハンドル。
<i>level</i>	情報を取得するコールスタックのレベル。

Returns

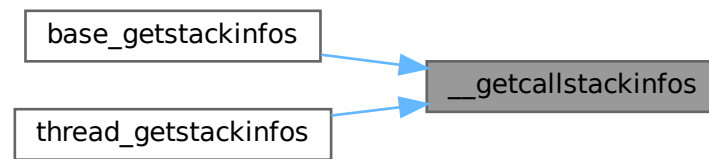
成功した場合は 1、失敗した場合は 0 を返します。

Definition at line 206 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00207 {
00208     SQStackInfos si;
00209     SQInteger seq = 0;
00210     const SQChar *name = NULL;
00211
00212     if (SQ_SUCCEEDED(sq_stackinfos(v, level, &si)))
00213     {
00214         const SQChar *fn = _SC("unknown");
00215         const SQChar *src = _SC("unknown");
00216         if (si.funcname) fn = si.funcname;
00217         if (si.source) src = si.source;
00218         sq_newtable(v);
00219         sq_pushstring(v, _SC("func"), -1);
00220         sq_pushstring(v, fn, -1);
00221         sq_newslot(v, -3, SQFalse);
00222         sq_pushstring(v, _SC("src"), -1);
00223         sq_pushstring(v, src, -1);
00224         sq_newslot(v, -3, SQFalse);
00225         sq_pushstring(v, _SC("line"), -1);
00226         sq_pushinteger(v, si.line);
00227         sq_newslot(v, -3, SQFalse);
00228         sq_pushstring(v, _SC("locals"), -1);
00229         sq_newtable(v);
00230         seq=0;
00231         while ((name = sq_getlocal(v, level, seq))) {
00232             sq_pushstring(v, name, -1);
00233             sq_push(v, -2);
00234             sq_newslot(v, -4, SQFalse);
00235             sq_pop(v, 1);
00236             seq++;
00237         }
00238         sq_newslot(v, -3, SQFalse);
00239         return 1;
00240     }
00241
00242     return 0;
00243 }
```

Referenced by [base_getstackinfos\(\)](#), and [thread_getstackinfos\(\)](#).

Here is the caller graph for this function:



2.1.2.2 __map_array()

```
static SQInteger __map_array (
    SQArray * dest,
    SQArray * src,
    HSQUIRRELVM v ) [static]
```

配列の各要素に関数を適用し、結果を別の配列に格納するヘルパー関数。

array_map と array_apply の内部ロジック。

Parameters

<i>dest</i>	結果を格納する先の配列。
<i>src</i>	処理対象のソース配列。
<i>v</i>	Squirrel VM のハンドル。

Returns

成功した場合は 0、失敗した場合は SQ_ERROR。

Definition at line 1008 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```

1008 {
1009     SQObjectPtr temp;
1010     SQInteger size = src->Size();
1011     SQObject &closure = stack_get(v, 2);
1012     v->Push(closure);
1013
1014     SQInteger nArgs = 0;
1015     if(sq_type(closure) == OT_CLOSURE) {
1016         nArgs = _closure(closure)->_function->_nparameters;
1017     }
1018     else if (sq_type(closure) == OT_NATIVECLOSURE) {
1019         SQInteger nParamsCheck = _nativeclosure(closure)->_nparamscheck;
1020         if (nParamsCheck > 0)
1021             nArgs = nParamsCheck;
1022         else // push all params when there is no check or only minimal count set
1023             nArgs = 4;
1024     }
1025
1026     for(SQInteger n = 0; n < size; n++) {
1027         src->Get(n,temp);
1028         v->Push(src);
1029         v->Push(temp);
1030         if (nArgs >= 3)
  
```



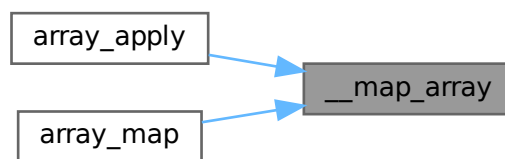
```

01031         v->Push(SQObjectPtr(n));
01032         if (nArgs >= 4)
01033             v->Push(src);
01034         if (SQ_FAILED(sq_call(v, nArgs, SQTrue, SQFalse))) {
01035             return SQ_ERROR;
01036         }
01037         dest->Set(n, v->GetUp(-1));
01038         v->Pop();
01039     }
01040     v->Pop();
01041     return 0;
01042 }

```

Referenced by [array_apply\(\)](#), and [array_map\(\)](#).

Here is the caller graph for this function:



2.1.2.3 _closure_acall()

```

static SQInteger _closure_acall (
    HSQUIRRELVM v,
    SQBool raiseerror ) [static]

```

配列を引数リストとしてクロージャを呼び出すヘルパー関数。

acall と pacall の内部実装。

Parameters

<i>v</i>	Squirrel VM のハンドル。
<i>raiseerror</i>	エラー発生時にVM エラーを発生させるかどうかのフラグ。

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1520 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

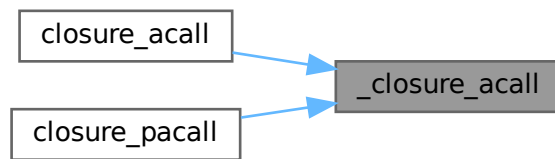
```

01521 {
01522     SQArray *aparams=_array(stack_get(v,2));
01523     SQInteger nparams=aparams->Size();
01524     v->Push(stack_get(v,1));
01525     for (SQInteger i=0;i<nparams;i++) v->Push(aparams->_values[i]);
01526     return SQ_SUCCEEDED(sq_call(v, nparams, SQTrue, raiseerror)) ? 1 : SQ_ERROR;
01527 }

```

Referenced by [closure_acall\(\)](#), and [closure_pacall\(\)](#).

Here is the caller graph for this function:



2.1.2.4 _hsort()

```

static bool _hsort (
    HSQUIRRELVm v,
    SQObjectPtr & arr,
    SQInteger SQ_UNUSED_ARG l,
    SQInteger SQ_UNUSED_ARG r,
    SQInteger func ) [static]
  
```

ヒープソートアルゴリズムの実装。

配列をインプレースでソートします。まず配列全体をヒープ化し、その後、ルート要素（最大値）を末尾に移動させ、残りの要素でヒープを再構築する、という処理を繰り返します。

Parameters

<i>v</i>	Squirrel VM のハンドル。
<i>arr</i>	ソート対象の配列オブジェクト。
<i>l</i>	未使用のパラメータ。
<i>r</i>	未使用のパラメータ。
<i>func</i>	カスタム比較関数のスタックインデックス。負の場合はデフォルト比較。

Returns

成功した場合は true、失敗した場合は false。

Definition at line 1284 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

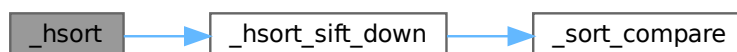
```

1285 {
1286     SQArray *a = _array(arr);
1287     SQInteger i;
1288     SQInteger array_size = a->Size();
1289     for (i = (array_size / 2); i >= 0; i--) {
1290         if(!_hsort_sift_down(v,a, i, array_size - 1,func)) return false;
1291     }
1292
1293     for (i = array_size-1; i >= 1; i--)
1294     {
1295         _Swap(a->values[0],a->values[i]);
1296         if(!_hsort_sift_down(v,a, 0, i-1,func)) return false;
1297     }
1298     return true;
1299 }
  
```

References [_hsort_sift_down\(\)](#).

Referenced by [array_sort\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



2.1.2.5 _hsort_sift_down()

```

static bool _hsort_sift_down (
    HSQUIRRELVm v,
    SQArray * arr,
    SQInteger root,
    SQInteger bottom,
    SQInteger func ) [static]
  
```

ヒープソートのヘルパー関数。ヒープのプロパティを維持します。

指定されたルートノードがヒープのプロパティを満たすように、要素を下に移動させます。親ノードが子ノードより小さくなるように（または比較関数に従って）要素を交換します。

Parameters

<i>v</i>	Squirrel VM のハンドル。
<i>arr</i>	操作対象の配列。
<i>root</i>	ヒープ化するサブツリーのルートノードのインデックス。
<i>bottom</i>	ヒープの最後の要素のインデックス。
<i>func</i>	カスタム比較関数のスタックインデックス。負の場合はデフォルト比較。

Returns

成功した場合は true、失敗した場合は false。

Definition at line 1231 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```

01232 {
01233     SQInteger maxChild;
01234     SQInteger done = 0;
01235     SQInteger ret;
01236     SQInteger root2;
01237     while (((root2 = root * 2) <= bottom) && (!done))
01238     {
01239         if (root2 == bottom) {
01240             maxChild = root2;
01241         }
01242         else {
01243             if (!_sort_compare(v, arr, arr->_values[root2], arr->_values[root2 + 1], func, ret))
01244                 return false;
01245             if (ret > 0) {
01246                 maxChild = root2;
01247             }
01248             else {
01249                 maxChild = root2 + 1;
01250             }
01251         }
01252
01253         if (!_sort_compare(v, arr, arr->_values[root], arr->_values[maxChild], func, ret))
01254             return false;
01255         if (ret < 0) {
01256             if (root == maxChild) {
01257                 v->Raise_Error(_SC("inconsistent compare function"));
01258                 return false; // We'd be swapping ourselves. The compare function is incorrect
01259             }
01260
01261             _Swap(arr->_values[root], arr->_values[maxChild]);
01262             root = maxChild;
01263         }
01264         else {
01265             done = 1;
01266         }
01267     }
01268     return true;
01269 }

```

References [_sort_compare\(\)](#).

Referenced by [_hsort\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



2.1.2.6 _sort_compare()

```
static bool _sort_compare (
    HSQUIRRELVm v,
    SQArray * arr,
    SQObjectPtr & a,
    SQObjectPtr & b,
    SQInteger func,
    SQInteger & ret ) [static]
```

ソート用の比較ヘルパー関数。

2つのオブジェクトを比較します。カスタム比較関数が提供されている場合はそれを使用し、そうでない場合はVMのデフォルトの比較ロジック (ObjCmp) を使用します。ソート中に配列が変更された場合はエラーを検出します。

Parameters

	<i>v</i>	Squirrel VM のハンドル。
	<i>arr</i>	ソート対象の配列。
	<i>a</i>	比較する最初のオブジェクト。
	<i>b</i>	比較する 2 番目のオブジェクト。
	<i>func</i>	カスタム比較関数のスタックインデックス。負の場合はデフォルト比較。
out	<i>ret</i>	比較結果 (<0, 0, >0) を格納する整数。

Returns

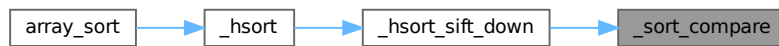
比較に成功した場合は `true`、失敗した場合は `false`。

Definition at line 1187 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01188 {
01189     if(func < 0) {
01190         if(!v->ObjCmp(a,b,ret)) return false;
01191     }
01192     else {
01193         SQInteger top = sq_gettop(v);
01194         sq_push(v, func);
01195         sq_pushroottable(v);
01196         v->Push(a);
01197         v->Push(b);
01198         SQObjectPtr *valptr = arr->_values._vals;
01199         SQUnsignedInteger precallsize = arr->_values.size();
01200         if(SQ_FAILED(sq_call(v, 3, SQTrue, SQFalse))) {
01201             if(!sq_isstring( v->_lasterror))
01202                 v->Raise_Error(_SC("compare func failed"));
01203             return false;
01204         }
01205         if(SQ_FAILED(sq_getinteger(v, -1, &ret))) {
01206             v->Raise_Error(_SC("numeric value expected as return value of the compare function"));
01207             return false;
01208         }
01209         if (precallsize != arr->_values.size() || valptr != arr->_values._vals) {
01210             v->Raise_Error(_SC("array resized during sort operation"));
01211             return false;
01212         }
01213         sq_settop(v, top);
01214         return true;
01215     }
01216     return true;
01217 }
```

Referenced by [_hsort_sift_down\(\)](#).

Here is the caller graph for this function:



2.1.2.7 array_append()

```
static SQInteger array_append (
    HSQUIRRELVM v ) [static]
```

配列の末尾に要素を追加します。

append() および push() メソッドの実装。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合は SQ_ERROR。

Definition at line 882 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00883 {
00884     return SQ_SUCCEEDED(sq_arrayappend(v,-2)) ? 1 : SQ_ERROR;
00885 }
```

2.1.2.8 array_apply()

```
static SQInteger array_apply (
    HSQUIRRELVM v ) [static]
```

配列の各要素に関数を適用し、配列をインプレースで変更します。

apply() メソッドの実装。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 1。エラー発生時は SQ_ERROR。

Definition at line 1067 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

01068 {
01069     SQObject &o = stack_get(v,1);
01070     if(SQ_FAILED(__map_array(_array(o),_array(o),v)))
01071         return SQ_ERROR;
01072     sq_pop(v,1);
01073     return 1;
01074 }

```

References [__map_array\(\)](#).

Here is the call graph for this function:



2.1.2.9 array_extend()

```

static SQInteger array_extend (
    HSQUIRRELVm v ) [static]

```

配列に別の配列の全要素を追加します。

extend() メソッドの実装。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

常に 1。

Definition at line 893 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

00894 {
00895     _array(stack_get(v,1))->Extend(_array(stack_get(v,2)));
00896     sq_pop(v,1);
00897     return 1;
00898 }

```

2.1.2.10 array_filter()

```

static SQInteger array_filter (
    HSQUIRRELVm v ) [static]

```

配列の要素をフィルタリングします。

filter() メソッドの実装。各要素に対して述語関数を呼び出し、その結果が true であった要素のみを含む新しい配列を生成します。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

常に 1。結果の配列をスタックにプッシュします。エラー発生時はSQ_ERROR。

Definition at line 1125 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

01126 {
01127     SQObject &o = stack_get(v,1);
01128     SQArray *a = _array(o);
01129     SQObjectPtr ret = SQArray::Create(_ss(v),0);
01130     SQInteger size = a->Size();
01131     SQObjectPtr val;
01132     for(SQInteger n = 0; n < size; n++) {
01133         a->Get(n,val);
01134         v->Push(o);
01135         v->Push(n);
01136         v->Push(val);
01137         if(SQ_FAILED(sq_call(v,3,SQTrue,SQFalse))) {
01138             return SQ_ERROR;
01139         }
01140         if(!SQVM::IsFalse(v->GetUp(-1))) {
01141             _array(ret)->Append(val);
01142         }
01143         v->Pop();
01144     }
01145     v->Push(ret);
01146     return 1;
01147 }
```

2.1.2.11 array_find()

```

static SQInteger array_find (
    HSQUIRRELVm v ) [static]
```

配列内から指定された値を検索し、最初のインデックスを返します。

find() メソッドの実装。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

成功した場合は 1（インデックスをスタックにプッシュ）。見つからない場合は 0。

Definition at line 1155 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

01156 {
01157     SQObject &o = stack_get(v,1);
01158     SQObjectPtr &val = stack_get(v,2);
01159     SQArray *a = _array(o);
01160     SQInteger size = a->Size();
01161     SQObjectPtr temp;
01162     for(SQInteger n = 0; n < size; n++) {
01163         bool res = false;
01164         a->Get(n,temp);
01165         if(SQVM::IsEqual(temp,val,res) && res) {
01166             v->Push(n);
01167             return 1;
01168         }
01169     }
01170     return 0;
01171 }
```


2.1.2.12 array_insert()

```
static SQInteger array_insert (
    HSQUIRRELVM v ) [static]
```

配列の指定したインデックスに要素を挿入します。

insert() メソッドの実装。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 1。インデックスが範囲外の場合はエラーをスローします。

Definition at line 944 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00945 {
00946     SQObject &o=stack_get(v,1);
00947     SQObject &idx=stack_get(v,2);
00948     SQObject &val=stack_get(v,3);
00949     if(!_array(o)->Insert(tointeger(idx),val))
00950         return sq_throwerror(v,_SC("index out of range"));
00951     sq_pop(v,2);
00952     return 1;
00953 }
```

2.1.2.13 array_map()

```
static SQInteger array_map (
    HSQUIRRELVM v ) [static]
```

配列の各要素を変換（マップ）し、新しい配列を返します。

map() メソッドの実装。配列の各要素に関数を適用し、その戻り値からなる新しい配列を生成します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 1。結果の配列をスタックにプッシュします。エラー発生時はSQ_ERROR。

Definition at line 1050 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01051 {
01052     SQObject &o = stack_get(v,1);
01053     SQInteger size = _array(o)->Size();
01054     SQObjectPtr ret = SQArray::Create(_ss(v),size);
01055     if(SQ_FAILED(__map_array(_array(ret),_array(o),v)))
01056         return SQ_ERROR;
01057     v->Push(ret);
01058     return 1;
01059 }
```

References [__map_array\(\)](#).

Here is the call graph for this function:



2.1.2.14 array_pop()

```
static SQInteger array_pop (
    HSQUIRRELVM v ) [static]
```

配列の末尾の要素を削除し、その要素を返します。

pop() メソッドの実装。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 917 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00918 {
00919     return SQ_SUCCEEDED(sq_arraypop(v,1,SQTrue)) ? 1 : SQ_ERROR;
00920 }
```

2.1.2.15 array_reduce()

```
static SQInteger array_reduce (
    HSQUIRRELVM v ) [static]
```

配列の要素を単一の値に集約（リデュース）します。

reduce() メソッドの実装。アキュムレータと配列の各要素に対して関数を適用し、結果を累積します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 1。集約結果をスタックにプッシュします。エラー発生時はSQ_ERROR。

Definition at line 1082 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

01083 {
01084     SQObject &o = stack_get(v,1);
01085     SQArray *a = _array(o);
01086     SQInteger size = a->Size();
01087     SQObjectPtr res;
01088     SQInteger iterStart;
01089     if (sq_gettop(v)>2) {
01090         res = stack_get(v,3);
01091         iterStart = 0;
01092     } else if (size==0) {
01093         return 0;
01094     } else {
01095         a->Get(0,res);
01096         iterStart = 1;
01097     }
01098     if (size > iterStart) {
01099         SQObjectPtr other;
01100         v->Push(stack_get(v,2));
01101         for (SQInteger n = iterStart; n < size; n++) {
01102             a->Get(n,other);
01103             v->Push(o);
01104             v->Push(res);
01105             v->Push(other);
01106             if (SQ_FAILED(sq_call(v,3,SQTrue,SQFalse))) {
01107                 return SQ_ERROR;
01108             }
01109             res = v->GetUp(-1);
01110             v->Pop();
01111         }
01112         v->Pop();
01113     }
01114     v->Push(res);
01115     return 1;
01116 }

```

2.1.2.16 array_remove()

```

static SQInteger array_remove (
    HSQUIRRELVN v ) [static]

```

配列の指定したインデックスの要素を削除し、その要素を返します。

remove() メソッドの実装。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

成功した場合は 1。インデックスが範囲外の場合はエラーをスローします。

Definition at line 961 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

00962 {
00963     SQObject &o = stack_get(v, 1);
00964     SQObject &idx = stack_get(v, 2);
00965     if (!sq_isnumeric(idx)) return sq_throwerror(v, _SC("wrong type"));
00966     SQObjectPtr val;
00967     if (_array(o)->Get(tointeger(idx), val)) {
00968         _array(o)->Remove(tointeger(idx));
00969         v->Push(val);
00970         return 1;
00971     }
00972     return sq_throwerror(v, _SC("idx out of range"));
00973 }

```

2.1.2.17 array_resize()

```

static SQInteger array_resize (
    HSQUIRRELVN v ) [static]

```

配列のサイズを変更します。

`resize()` メソッドの実装。サイズを大きくする場合、オプションで指定された値で新しい要素を埋めます。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

常に 1。サイズに負の値を指定した場合などはエラーをスローします。

Definition at line 981 of file `squirrel3_squirrel_sqbaselib_v1.cpp`.

```
00982 {
00983     SQObject &o = stack_get(v, 1);
00984     SQObject &nsiz = stack_get(v, 2);
00985     SQObjectPtr fill;
00986     if(sq_isnumeric(nsiz)) {
00987         SQInteger sz = tointeger(nsiz);
00988         if (sz<0)
00989             return sq_throwerror(v, _SC("resizing to negative length"));
00990
00991         if(sq_gettop(v) > 2)
00992             fill = stack_get(v, 3);
00993         _array(o)->Resize(sz,fill);
00994         sq_settop(v, 1);
00995         return 1;
00996     }
00997     return sq_throwerror(v, _SC("size must be a number"));
00998 }
```

2.1.2.18 array_reverse()

```
static SQInteger array_reverse (
    HSQUIRRELVm v ) [static]
```

配列の要素の順序を反転させます。

`reverse()` メソッドの実装。配列をインプレースで変更します。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

成功した場合は 1、失敗した場合は `SQ_ERROR`。

Definition at line 906 of file `squirrel3_squirrel_sqbaselib_v1.cpp`.

```
00907 {
00908     return SQ_SUCCEEDED(sq_arrayreverse(v,-1)) ? 1 : SQ_ERROR;
00909 }
```

2.1.2.19 array_slice()

```
static SQInteger array_slice (
    HSQUIRRELVm v ) [static]
```

配列の一部を切り出した新しい配列（スライス）を返します。

`slice()` メソッドの実装。開始インデックスと終了インデックスを指定します。

Parameters

v	Squirrel VM のハンドル。
-----	--------------------

Returns

常に 1。結果の配列をスタックにプッシュします。インデックスが不正な場合はエラー。

Definition at line 1328 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

01329 {
01330     SQInteger sidx,eidx;
01331     SQObjectPtr o;
01332     if(get_slice_params(v,sidx,eidx,o)==-1) return -1;
01333     SQInteger alen = _array(o)->Size();
01334     if(sidx < 0)sidx = alen + sidx;
01335     if(eidx < 0)eidx = alen + eidx;
01336     if(eidx < sidx) return sq_throwerror(v,_SC("wrong indexes"));
01337     if(eidx > alen || sidx < 0) return sq_throwerror(v,_SC("slice out of range"));
01338     SQArray *arr=SQArray::Create(_ss(v),eidx-sidx);
01339     SQObjectPtr t;
01340     SQInteger count=0;
01341     for(SQInteger i=sidx;i<eidx;i++){
01342         _array(o)->Get(i,t);
01343         arr->Set(count++,t);
01344     }
01345     v->Push(arr);
01346     return 1;
01347 }
01348 }
```

References [get_slice_params\(\)](#).

Here is the call graph for this function:



2.1.2.20 array_sort()

```

static SQInteger array_sort (
    HSQUIRRELVm v ) [static]
```

配列をソートします。

sort() メソッドの実装。ヒープソートアルゴリズムを使用します。オプションでカスタム比較関数を指定できます。

Parameters

v	Squirrel VM のハンドル。
-----	--------------------

Returns

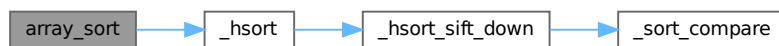
成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1308 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
01309 {
01310     SQInteger func = -1;
01311     SQObjectPtr &o = stack_get(v,1);
01312     if(_array(o)->Size() > 1) {
01313         if(sq_gettop(v) == 2) func = 2;
01314         if(!_hsort(v, o, 0, _array(o)->Size()-1, func))
01315             return SQ_ERROR;
01316     }
01317     sq_settop(v,1);
01318     return 1;
01319 }
01320 }
```

References [_hsort\(\)](#).

Here is the call graph for this function:



2.1.2.21 array_top()

```
static SQInteger array_top (
    HSQUIRRELVM v ) [static]
```

配列の末尾の要素を削除せずに返します。

`top()` メソッドの実装。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

成功した場合は 1。配列が空の場合はエラーをスローします。

Definition at line 928 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
00929 {
00930     SQObject &o=stack_get(v,1);
00931     if(_array(o)->Size()>0) {
00932         v->Push(_array(o)->Top());
00933         return 1;
00934     }
00935     else return sq_throwerror(v,_SC("top() on a empty array"));
00936 }
```

2.1.2.22 base_array()

```
static SQInteger base_array (
    HSQUIRRELVM v ) [static]
```

新しい配列を生成します。

指定されたサイズの新しい配列を生成します。オプションで初期値を指定できます。生成した配列をスタックにプッシュします。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1。

Definition at line 422 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
00423 {
00424     SQArray *a;
00425     SQObject &size = stack_get(v,2);
00426     if(sq_gettop(v) > 2) {
00427         a = SQArray::Create(_ss(v),0);
00428         a->Resize(tointeger(size),stack_get(v,3));
00429     }
00430     else {
00431         a = SQArray::Create(_ss(v),tointeger(size));
00432     }
00433     v->Push(a);
00434     return 1;
00435 }
```

2.1.2.23 base_assert()

```
static SQInteger base_assert (
    HSQUIRRELVm v ) [static]
```

アサーションを評価します。

スタックの 2 番目にある値が `false` の場合、アサーション失敗エラーをスローします。オプションの第 2 引数（スタックの 3 番目）をエラーメッセージとして使用できます。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

条件が `true` の場合は 0。 `false` の場合はエラーをスローするため戻りません。

Definition at line 268 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
00269 {
00270     if(SQVM::IsFalse(stack_get(v,2))){
00271         SQInteger top = sq_gettop(v);
00272         if (top>2 && SQ_SUCCEEDED(sq_tostring(v,3))) {
00273             const SQChar *str = 0;
00274             if (SQ_SUCCEEDED(sq_getstring(v,-1,&str))) {
00275                 return sq_throwerror(v, str);
00276             }
00277         }
00278         return sq_throwerror(v, _SC("assertion failed"));
00279     }
00280     return 0;
00281 }
```

2.1.2.24 base_callee()

```
static SQInteger base_callee (
    HSQUIRRELVM v ) [static]
```

現在実行中の関数の呼び出し元クロージャを取得します。

コールスタックの一つ前のレベルにあるクロージャ（呼び出し元）を取得し、スタックにプッシュします。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1。コールスタックが 1 レベルしかない場合はエラーをスローします。

Definition at line 460 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00461 {
00462     if(v->_callsstacksize > 1)
00463     {
00464         v->Push(v->_callsstack[v->_callsstacksize - 2]._closure);
00465         return 1;
00466     }
00467     return sq_throwerror(v, _SC("no closure in the calls stack"));
00468 }
```

2.1.2.25 base_collectgarbage()

```
static SQInteger base_collectgarbage (
    HSQUIRRELVM v ) [static]
```

ガーベッジコレクタを実行します。

到達不可能なオブジェクトを解放し、メモリを回収します。回収されたオブジェクトの数をスタックにプッシュします。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

スタックにプッシュした値の数 (1)。

Definition at line 81 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00082 {
00083     sq_pushinteger(v, sq_collectgarbage(v));
00084     return 1;
00085 }
```

2.1.2.26 base_compilestring()

```
static SQInteger base_compilestring (
    HSQUIRRELVM v ) [static]
```


文字列をコンパイルしてクロージャを生成します。

与えられたSquirrel コードの文字列をコンパイルし、結果として得られる クロージャをスタックにプッシュします。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 368 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
00369 {
00370     SQInteger nargs=sq_gettop(v);
00371     const SQChar *src=NULL, *name=_SC("unnamedbuffer");
00372     SQInteger size;
00373     sq_getstring(v,2,&src);
00374     size=sq_getsize(v,2);
00375     if(nargs>2){
00376         sq_getstring(v,3,&name);
00377     }
00378     if(SQ_SUCCEEDED(sq_compilebuffer(v,src,size,name,SQFalse)))
00379         return 1;
00380     else
00381         return SQ_ERROR;
00382 }
```

2.1.2.27 base_dummy()

```
static SQInteger base_dummy (
    HSQUIRRELVm SQ_UNUSED_ARGv ) [static]
```

何も行わないダミー関数です。

この関数はプレースホルダーやテスト目的で使用されることがあります。常に 0 を返します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 0。

Definition at line 67 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
00068 {
00069     return 0;
00070 }
```

2.1.2.28 base_enableddebuginfo()

```
static SQInteger base_enableddebuginfo (
    HSQUIRRELVm v ) [static]
```

デバッグ情報の生成を有効または無効にします。

引数（スタックの 2 番目）が false でない場合、デバッグ情報の生成を有効にします。これにより、ファイル名や行番号などの情報が実行時に利用可能になります。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

常に 0。

Definition at line 189 of file `squirrel3_squirrel_sqbaselib_v1.cpp`.

```
00190 {
00191     SQObjectPtr &o=stack_get(v,2);
00192
00193     sq_enableddebuginfo(v,SQVM::IsFalse(o)?SQFalse:SQTrue);
00194     return 0;
00195 }
```

2.1.2.29 base_error()

```
static SQInteger base_error (
    HSQUIRRELVm v ) [static]
```

引数をエラー出力します。

スタックの 2 番目にある値を文字列に変換し、VM に登録された エラーコールバック関数 (`_errorfunc`) を使って出力します。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

成功した場合は 0、失敗した場合は `SQ_ERROR`。

Definition at line 347 of file `squirrel3_squirrel_sqbaselib_v1.cpp`.

```
00348 {
00349     const SQChar *str;
00350     if(SQ_SUCCEEDED(sq_tostring(v,2)))
00351     {
00352         if(SQ_SUCCEEDED(sq_getstring(v,-1,&str))) {
00353             if(_ss(v)->_errorfunc) _ss(v)->_errorfunc(v,_SC("%s"),str);
00354             return 0;
00355         }
00356     }
00357     return SQ_ERROR;
00358 }
```

2.1.2.30 base_getconsttable()

```
static SQInteger base_getconsttable (
    HSQUIRRELVm v ) [static]
```

定数テーブルをスタックにプッシュします。

VM の現在の定数テーブルを取得し、スタックのトップに配置します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

スタックにプッシュした値の数 (1)。

Definition at line 119 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00120 {  
00121     v->Push(_ss(v)->_consts);  
00122     return 1;  
00123 }
```

2.1.2.31 base_getroottable()

```
static SQInteger base_getroottable (  
    HSQUIRRELVm v ) [static]
```

ルートテーブルをスタックにプッシュします。

VM の現在のルートテーブルを取得し、スタックのトップに配置します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

スタックにプッシュした値の数 (1)。

Definition at line 107 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00108 {  
00109     v->Push(v->_roottable);  
00110     return 1;  
00111 }
```

2.1.2.32 base_getstackinfos()

```
static SQInteger base_getstackinfos (  
    HSQUIRRELVm v ) [static]
```

getstackinfos 関数のSquirrel API ラッパー。

スタックからレベルを示す整数を取得し、__getcallstackinfos を呼び出して スタック情報を取得し、その結果を返します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

`__getcallstackinfos` の戻り値。

Definition at line 253 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00254 {
00255     SQInteger level;
00256     sq_getinteger(v, -1, &level);
00257     return __getcallstackinfos(v, level);
00258 }
```

References [__getcallstackinfos\(\)](#).

Here is the call graph for this function:

**2.1.2.33 base_newthread()**

```
static SQInteger base_newthread (
    HSQUIRRELVm v ) [static]
```

新しいスレッドを生成します。

引数として与えられたクロージャを開始点とする新しいVM スレッドを生成し、スタックにプッシュします。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

成功した場合は 1。

Definition at line 392 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00393 {
00394     SQObjectPtr &func = stack_get(v, 2);
00395     SQInteger stksize = (_closure(func)->_function->_stacksize << 1) + 2;
00396     HSQUIRRELVm newv = sq_newthread(v, (stksize < MIN_STACK_OVERHEAD + 2)? MIN_STACK_OVERHEAD + 2 :
    stksize);
00397     sq_move(newv, v, -2);
00398     return 1;
00399 }
```

2.1.2.34 base_print()

```
static SQInteger base_print (
    HSQUIRRELVm v ) [static]
```

引数を標準出力に出力します。

スタックの 2 番目にある値を文字列に変換し、VM に登録された プリントコールバック関数 (`_printfunc`) を使って出力します。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

成功した場合は 0、失敗した場合は `SQ_ERROR`。

Definition at line 326 of file `squirrel3_squirrel_sqbaselib_v1.cpp`.

```
00327 {
00328     const SQChar *str;
00329     if (SQ_SUCCEEDED(sq_tostring(v, 2)))
00330     {
00331         if (SQ_SUCCEEDED(sq_getstring(v, -1, &str))) {
00332             if (_ss(v) -> _printfunc) _ss(v) -> _printfunc(v, _SC("%s"), str);
00333             return 0;
00334         }
00335     }
00336     return SQ_ERROR;
00337 }
```

2.1.2.35 base_resurrectureachable()

```
static SQInteger base_resurrectureachable (
    HSQUIRRELVM v ) [static]
```

到達不能オブジェクトを復活させます。

ガーベッジコレクタによって到達不能と判断されたオブジェクトを、再び到達可能に戻します。主にデバッグ目的で使用されます。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

常に 1。

Definition at line 94 of file `squirrel3_squirrel_sqbaselib_v1.cpp`.

```
00095 {
00096     sq_resurrectunreachable(v);
00097     return 1;
00098 }
```

2.1.2.36 base_setconsttable()

```
static SQInteger base_setconsttable (
    HSQUIRRELVM v ) [static]
```

定数テーブルを設定します。

スタックトップにあるテーブルを新しい定数テーブルとして設定します。変更前の古い定数テーブルをスタックにプッシュして返します。

Parameters

v	Squirrel VM のハンドル。
-----	--------------------

Returns

成功した場合は古い定数テーブルをプッシュし 1 を返す。失敗した場合はSQ_ERROR。

Definition at line 149 of file `squirrel3_squirrel_sqbaselib_v1.cpp`.

```
00150 {
00151     SQObjectPtr o = _ss(v)->_consts;
00152     if (SQ_FAILED(sq_setconsttable(v))) return SQ_ERROR;
00153     v->Push(o);
00154     return 1;
00155 }
```

2.1.2.37 base_setdebughook()

```
static SQInteger base_setdebughook (
    HSQUIRRELVM v ) [static]
```

新しいデバッグフックを設定します。

スタックトップのクロージャをVMの新しいデバッグフックとして設定します。

Parameters

v	Squirrel VM のハンドル。
-----	--------------------

Returns

常に 0。

Definition at line 175 of file `squirrel3_squirrel_sqbaselib_v1.cpp`.

```
00176 {
00177     sq_setdebughook(v);
00178     return 0;
00179 }
```

2.1.2.38 base_seterrorhandler()

```
static SQInteger base_seterrorhandler (
    HSQUIRRELVM v ) [static]
```

新しいエラーハンドラを設定します。

スタックトップのクロージャをVMの新しいエラーハンドラとして設定します。

Parameters

v	Squirrel VM のハンドル。
-----	--------------------

Returns

常に 0。

Definition at line 163 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
00164 {
00165     sq_seterrorhandler(v);
00166     return 0;
00167 }
```

2.1.2.39 base_setroottable()

```
static SQInteger base_setroottable (
    HSQUIRRELVM v ) [static]
```

ルートテーブルを設定します。

スタックトップにあるテーブルを新しいルートテーブルとして設定します。変更前の古いルートテーブルをスタックにプッシュして返します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は古いルートテーブルをプッシュし 1 を返す。失敗した場合は SQ_ERROR。

Definition at line 133 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
00134 {
00135     SQObjectPtr o = v->_roottable;
00136     if (SQ_FAILED(sq_setroottable(v))) return SQ_ERROR;
00137     v->Push(o);
00138     return 1;
00139 }
```

2.1.2.40 base_suspend()

```
static SQInteger base_suspend (
    HSQUIRRELVM v ) [static]
```

現在の VM を中断（サスペンド）します。

VM の実行を中断し、呼び出し元に制御を返します。ジェネレータやコルーチンを実現するために使用されます。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

sq_suspendvm の結果。

Definition at line 409 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00410 {
00411     return sq_suspendvm(v);
00412 }
```

2.1.2.41 base_type()

```
static SQInteger base_type (
    HSQUIRRELVM v ) [static]
```

オブジェクトの型名を取得します。

引数として与えられたオブジェクトの型名を表す文字列を スタックにプッシュします。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

成功した場合は 1。

Definition at line 445 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00446 {
00447     SQObjectPtr &o = stack_get(v,2);
00448     v->Push(SQString::Create(_ss(v),GetTypeName(o),-1));
00449     return 1;
00450 }
```

2.1.2.42 class_getattributes()

```
static SQInteger class_getattributes (
    HSQUIRRELVM v ) [static]
```

クラスメンバの属性を取得します。

getattributes() メソッドの実装。指定されたキー（メンバ名）に対応する属性テーブルを取得します。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1884 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01885 {
01886     return SQ_SUCCEEDED(sq_getattributes(v,-2)) ? 1 : SQ_ERROR;
01887 }
```


2.1.2.43 class_getbase()

```
static SQInteger class_getbase (
    HSQUIRRELVM v ) [static]
```

クラスの基底クラスを取得します。

getbase() メソッドの実装。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1917 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01918 {
01919     return SQ_SUCCEEDED(sq_getbase(v, -1)) ? 1 : SQ_ERROR;
01920 }
```

2.1.2.44 class_instance()

```
static SQInteger class_instance (
    HSQUIRRELVM v ) [static]
```

クラスの新しいインスタンスを生成します。

instance() メソッドの実装。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1906 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01907 {
01908     return SQ_SUCCEEDED(sq_createinstance(v, -1)) ? 1 : SQ_ERROR;
01909 }
```

2.1.2.45 class_newmember()

```
static SQInteger class_newmember (
    HSQUIRRELVM v ) [static]
```

クラスに新しいメンバを追加します。

newmember() メソッドの実装。_newmember メタメソッドをトリガーする可能性があります。

Parameters

v	Squirrel VM のハンドル。
----------	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1928 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

01929 {
01930     SQInteger top = sq_gettop(v);
01931     SQBool bstatic = SQFalse;
01932     if(top == 5)
01933     {
01934         sq_tobool(v,-1,&bstatic);
01935         sq_pop(v,1);
01936     }
01937
01938     if(top < 4) {
01939         sq_pushnull(v);
01940     }
01941     return SQ_SUCCEEDED(sq_newmember(v,-4,bstatic)) ? 1 : SQ_ERROR;
01942 }
```

2.1.2.46 class_rawnewmember()

```
static SQInteger class_rawnewmember (
    HSQUIRRELVm v ) [static]
```

クラスに新しいメンバをメタメソッドを無視して追加します。

rawnewmember() メソッドの実装。メタメソッドをトリガーせずにメンバを直接追加します。

Parameters

v	Squirrel VM のハンドル。
----------	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1950 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

01951 {
01952     SQInteger top = sq_gettop(v);
01953     SQBool bstatic = SQFalse;
01954     if(top == 5)
01955     {
01956         sq_tobool(v,-1,&bstatic);
01957         sq_pop(v,1);
01958     }
01959
01960     if(top < 4) {
01961         sq_pushnull(v);
01962     }
01963     return SQ_SUCCEEDED(sq_rawnewmember(v,-4,bstatic)) ? 1 : SQ_ERROR;
01964 }
```

2.1.2.47 class_setattributes()

```
static SQInteger class_setattributes (
    HSQUIRRELVm v ) [static]
```

クラスメンバの属性を設定します。

`setattributes()` メソッドの実装。指定されたキー（メンバ名）に新しい属性テーブルを設定します。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

成功した場合は 1、失敗した場合は `SQ_ERROR`。

Definition at line 1895 of file `squirrel3_squirrel_sqbaselib_v1.cpp`.

```
01896 {
01897     return SQ_SUCCEEDED(sq_setattributes(v,-3)) ? 1:SQ_ERROR;
01898 }
```

2.1.2.48 closure_acall()

```
static SQInteger closure_acall (
    HSQUIRRELVm v ) [static]
```

配列を引数リストとしてクロージャを呼び出します。

`acall()` メソッドの実装。エラーが発生すると VM エラーが発生させます。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

`_closure_acall` の戻り値。

Definition at line 1535 of file `squirrel3_squirrel_sqbaselib_v1.cpp`.

```
01536 {
01537     return _closure_acall(v,SQTrue);
01538 }
```

References `_closure_acall()`.

Here is the call graph for this function:



2.1.2.49 closure_bindenv()

```
static SQInteger closure_bindenv (
    HSQUIRRELVM v ) [static]
```

クロージャに環境オブジェクトを束縛します。

bindenv() メソッドの実装。クロージャの自由変数の解決に使われる環境（テーブルやクラスインスタンス）を設定します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1557 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01558 {
01559     if (SQ_FAILED(sq_bindenv(v,1)))
01560         return SQ_ERROR;
01561     return 1;
01562 }
```

2.1.2.50 closure_call()

```
static SQInteger closure_call (
    HSQUIRRELVM v ) [static]
```

クロージャを呼び出します。

call() メソッドの実装。可能であれば末尾呼び出し最適化を行います。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1503 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01504 {
01505     SQObjectPtr &c = stack_get(v, -1);
01506     if (sq_type(c) == OT_CLOSURE && (_closure(c)->_bgenerator == false))
01507     {
01508         return sq_tailcall(v, sq_gettop(v) - 1);
01509     }
01510     return SQ_SUCCEEDED(sq_call(v, sq_gettop(v) - 1, SQTrue, SQTrue)) ? 1 : SQ_ERROR;
01511 }
```

2.1.2.51 closure_getinfos()

```
static SQInteger closure_getinfos (
    HSQUIRRELVM v ) [static]
```

クロージャに関する情報を取得します。

getinfos() メソッドの実装。関数名、ソース名、パラメータ、デフォルトパラメータなどの情報を格納したテーブルを生成して返します。ネイティブクロージャとSquirrel クロージャの両方に対応しています。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 1。結果のテーブルをスタックにプッシュします。

Definition at line 1597 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```

1597     {
1598         SQObject o = stack_get(v,1);
1599         SQTable *res = SQTable::Create(_ss(v),4);
1600         if(sq_type(o) == OT_CLOSURE) {
1601             SQFunctionProto *f = _closure(o)->_function;
1602             SQInteger nparams = f->nparameters + (f->varparams?1:0);
1603             SQObjectPtr params = SQArray::Create(_ss(v),nparams);
1604             SQObjectPtr defparams = SQArray::Create(_ss(v),f->ndefaultparams);
1605             for(SQInteger n = 0; n<f->nparameters; n++) {
1606                 _array(params)->Set((SQInteger)n,f->parameters[n]);
1607             }
1608             for(SQInteger j = 0; j<f->ndefaultparams; j++) {
1609                 _array(defparams)->Set((SQInteger)j,_closure(o)->defaultparams[j]);
1610             }
1611             if(f->varparams) {
1612                 _array(params)->Set(nparams-1,SQString::Create(_ss(v),_SC("..."),-1));
1613             }
1614             res->NewSlot(SQString::Create(_ss(v),_SC("native"),-1),false);
1615             res->NewSlot(SQString::Create(_ss(v),_SC("name"),-1),f->_name);
1616             res->NewSlot(SQString::Create(_ss(v),_SC("src"),-1),f->_sourcename);
1617             res->NewSlot(SQString::Create(_ss(v),_SC("parameters"),-1),params);
1618             res->NewSlot(SQString::Create(_ss(v),_SC("varargs"),-1),f->varparams);
1619             res->NewSlot(SQString::Create(_ss(v),_SC("defparams"),-1),defparams);
1620         }
1621         else { //OT_NATIVECLOSURE
1622             SQNativeClosure *nc = _nativeclosure(o);
1623             res->NewSlot(SQString::Create(_ss(v),_SC("native"),-1),true);
1624             res->NewSlot(SQString::Create(_ss(v),_SC("name"),-1),nc->_name);
1625             res->NewSlot(SQString::Create(_ss(v),_SC("paramscheck"),-1),nc->_nparamscheck);
1626             SQObjectPtr typecheck;
1627             if(nc->_typecheck.size() > 0) {
1628                 typecheck =
1629                     SQArray::Create(_ss(v), nc->_typecheck.size());
1630                 for(SQUnsignedInteger n = 0; n<nc->_typecheck.size(); n++) {
1631                     _array(typecheck)->Set((SQInteger)n,nc->_typecheck[n]);
1632                 }
1633             }
1634             res->NewSlot(SQString::Create(_ss(v),_SC("typecheck"),-1),typecheck);
1635         }
1636         v->Push(res);
1637         return 1;
1638     }

```

2.1.2.52 closure_getroot()

```
static SQInteger closure_getroot (
    HSQUIRRELVM v ) [static]
```

クロージャのルートテーブルを取得します。

getroot() メソッドの実装。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1570 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01571 {  
01572     if (SQ_FAILED(sq_getclosuresroot(v,-1)))  
01573         return SQ_ERROR;  
01574     return 1;  
01575 }
```

2.1.2.53 closure_pacall()

```
static SQInteger closure_pacall (  
    HSQUIRRELVm v ) [static]
```

配列を引数リストとしてクロージャを保護付きで呼び出します。

pacall() メソッドの実装。エラーが発生してもVM エラーを発生させません。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

_closure_acall の戻り値。

Definition at line 1546 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01547 {  
01548     return _closure_acall(v,SQFalse);  
01549 }
```

References [_closure_acall\(\)](#).

Here is the call graph for this function:



2.1.2.54 closure_pcall()

```
static SQInteger closure_pcall (
    HSQUIRRELVM v ) [static]
```

クロージャを保護付きで呼び出します。

pcall() メソッドの実装。呼び出し中にエラーが発生してもスクリプトを停止せず、エラーオブジェクトを返します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1492 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01493 {
01494     return SQ_SUCCEEDED(sq_call(v, sq_gettop(v)-1, SQTrue, SQFalse)) ? 1 : SQ_ERROR;
01495 }
```

2.1.2.55 closure_setroot()

```
static SQInteger closure_setroot (
    HSQUIRRELVM v ) [static]
```

クロージャのルートテーブルを設定します。

setroot() メソッドの実装。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1583 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01584 {
01585     if(SQ_FAILED(sq_setclosureroor(v,-2)))
01586         return SQ_ERROR;
01587     return 1;
01588 }
```

2.1.2.56 container_rawexists()

```
static SQInteger container_rawexists (
    HSQUIRRELVM v ) [static]
```

コンテナ内にキーが存在するかをデリゲートを無視して確認します。

rawin() メソッドの実装。_get メタメソッドをトリガーせずにキーの存在を直接確認します。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

常に 1。結果 (true/false) をスタックにプッシュします。

Definition at line 691 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00692 {
00693     if (SQ_SUCCEEDED(sq_rawget(v,-2))) {
00694         sq_pushbool(v,SQTrue);
00695         return 1;
00696     }
00697     sq_pushbool(v,SQFalse);
00698     return 1;
00699 }
```

2.1.2.57 container_rawget()

```
static SQInteger container_rawget (
    HSQUIRRELVm v ) [static]
```

コンテナから値をデリゲートを無視して取得します。

rawget () メソッドの実装。_get メタメソッドをトリガーせずに値を直接取得します。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

成功した場合は 1、失敗した場合は SQ_ERROR。

Definition at line 718 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00719 {
00720     return SQ_SUCCEEDED(sq_rawget(v,-2)) ? 1 : SQ_ERROR;
00721 }
```

2.1.2.58 container_rawset()

```
static SQInteger container_rawset (
    HSQUIRRELVm v ) [static]
```

コンテナに値をデリゲートを無視して設定します。

rawset () メソッドの実装。_set メタメソッドをトリガーせずに値を直接設定します。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 707 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00708 {
00709     return SQ_SUCCEEDED(sq_rawset(v,-3)) ? 1 : SQ_ERROR;
00710 }
```

2.1.2.59 default_delegate_len()

```
static SQInteger default_delegate_len (
    HSQUIRRELVM v ) [static]
```

オブジェクトのサイズ（長さ）を取得するデフォルトデリゲート関数。

len() メソッドの実装。sq_getsize を呼び出し、結果をスタックにプッシュします。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 1。

Definition at line 544 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00545 {
00546     v->Push(SQInteger(sq_getsize(v,1)));
00547     return 1;
00548 }
```

2.1.2.60 default_delegate_tofloat()

```
static SQInteger default_delegate_tofloat (
    HSQUIRRELVM v ) [static]
```

オブジェクトを浮動小数点数に変換するデフォルトデリゲート関数。

tofloat() メソッドの実装。文字列、整数、浮動小数点数、真偽値を変換します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 1。変換できない場合はエラーをスローします。

Definition at line 556 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00557 {
00558     SQObjectPtr &o=stack_get(v,1);
00559     switch(sq_type(o)){
```

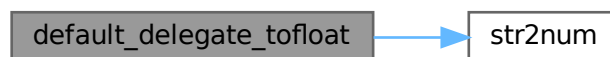
```

00560     case OT_STRING:{
00561         SQObjectPtr res;
00562         if(str2num(_stringval(o),res,10)){
00563             v->Push(SQObjectPtr(tofloat(res)));
00564             break;
00565         }}
00566         return sq_throwerror(v, _SC("cannot convert the string"));
00567         break;
00568     case OT_INTEGER:case OT_FLOAT:
00569         v->Push(SQObjectPtr(tofloat(o)));
00570         break;
00571     case OT_BOOL:
00572         v->Push(SQObjectPtr((SQFloat) (_integer(o)?1:0)));
00573         break;
00574     default:
00575         v->PushNull();
00576         break;
00577     }
00578     return 1;
00579 }

```

References [str2num\(\)](#).

Here is the call graph for this function:



2.1.2.61 default_delegate_tointeger()

```

static SQInteger default_delegate_tointeger (
    HSQUIRRELVm v ) [static]

```

オブジェクトを整数に変換するデフォルトデリゲート関数。

tointeger() メソッドの実装。文字列、整数、浮動小数点数、真偽値を変換します。文字列の場合、オプションで基数を指定できます。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

常に 1。変換できない場合はエラーをスローします。

Definition at line 588 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

00589 {
00590     SQObjectPtr &o=stack_get(v,1);
00591     SQInteger base = 10;
00592     if(sq_gettop(v) > 1) {
00593         sq_getinteger(v,2,&base);
00594     }
00595     switch(sq_type(o)){
00596     case OT_STRING:{

```

```

00597     SQObjectPtr res;
00598     if (str2num(_stringval(o), res, base)) {
00599         v->Push(SQObjectPtr(tointeger(res)));
00600         break;
00601     }
00602     return sq_throwerror(v, _SC("cannot convert the string"));
00603     break;
00604     case OT_INTEGER: case OT_FLOAT:
00605         v->Push(SQObjectPtr(tointeger(o)));
00606         break;
00607     case OT_BOOL:
00608         v->Push(SQObjectPtr(_integer(o) ? (SQInteger)1 : (SQInteger)0));
00609         break;
00610     default:
00611         v->PushNull();
00612         break;
00613     }
00614     return 1;
00615 }

```

References [str2num\(\)](#).

Here is the call graph for this function:



2.1.2.62 default_delegate_tostrng()

```

static SQInteger default_delegate_tostrng (
    HSQUIRRELVm v ) [static]

```

オブジェクトを文字列に変換するデフォルトデリゲート関数。

tostring() メソッドの実装。sq_tostrng API を呼び出して、スタック上のオブジェクトを文字列に変換します。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

成功した場合は 1、失敗した場合は SQ_ERROR。

Definition at line 623 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

00624 {
00625     if (SQ_FAILED(sq_tostrng(v, 1)))
00626         return SQ_ERROR;
00627     return 1;
00628 }

```

2.1.2.63 generator_getstatus()

```
static SQInteger generator_getstatus (
    HSQUIRRELVM v ) [static]
```

ジェネレータの現在の状態を取得します。

getstatus() メソッドの実装。状態は "suspended", "running", "dead" のいずれかです。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

常に 1。状態を表す文字列をスタックにプッシュします。

Definition at line 1664 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01665 {
01666     SQObject &o=stack_get(v,1);
01667     switch(_generator(o)->_state){
01668         case SQGenerator::eSuspended:v->Push(SQString::Create(_ss(v),_SC("suspended")));break;
01669         case SQGenerator::eRunning:v->Push(SQString::Create(_ss(v),_SC("running")));break;
01670         case SQGenerator::eDead:v->Push(SQString::Create(_ss(v),_SC("dead")));break;
01671     }
01672     return 1;
01673 }
```

2.1.2.64 get_slice_params()

```
static SQInteger get_slice_params (
    HSQUIRRELVM v,
    SQInteger &sidx,
    SQInteger &eidx,
    SQObjectPtr &o ) [static]
```

スライス操作のパラメータを取得します。

配列や文字列のスライス操作 (slice など) で使われる開始インデックスと 終了インデックスをスタックから取得するヘルパー関数です。

Parameters

	<i>v</i>	Squirrel VM のハンドル。
out	<i>sidx</i>	開始インデックス。
out	<i>eidx</i>	終了インデックス。
out	<i>o</i>	対象のオブジェクト。

Returns

常に 1。

Definition at line 294 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

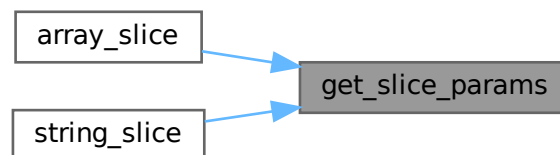
```

00295 {
00296     SQInteger top = sq_gettop(v);
00297     sidx=0;
00298     eidx=0;
00299     o=stack_get(v,1);
00300     if(top>1){
00301         SQObjectPtr &start=stack_get(v,2);
00302         if(sq_type(start)!=OT_NULL && sq_isnumeric(start)){
00303             sidx=tointeger(start);
00304         }
00305     }
00306     if(top>2){
00307         SQObjectPtr &end=stack_get(v,3);
00308         if(sq_isnumeric(end)){
00309             eidx=tointeger(end);
00310         }
00311     }
00312     else {
00313         eidx = sq_getsize(v,1);
00314     }
00315     return 1;
00316 }

```

Referenced by [array_slice\(\)](#), and [string_slice\(\)](#).

Here is the caller graph for this function:



2.1.2.65 instance_getclass()

```

static SQInteger instance_getclass (
    HSQUIRRELVM v ) [static]

```

インスタンスのクラスを取得します。

getclass() メソッドの実装。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1990 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

01991 {
01992     if(SQ_SUCCEEDED(sq_getclass(v,1)))
01993         return 1;
01994     return SQ_ERROR;
01995 }

```

2.1.2.66 number_delegate_tochar()

```
static SQInteger number_delegate_tochar (
    HSQUIRRELVM v ) [static]
```

数値を 1 文字の文字列に変換します。

tochar() メソッドの実装。数値を文字コードと解釈し、その文字を含む新しい文字列を生成します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 1。

Definition at line 659 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00660 {
00661     SQObject &o=stack_get(v,1);
00662     SQChar c = (SQChar)tointeger(o);
00663     v->Push(SQString::Create(_ss(v), (const SQChar *)&c,1));
00664     return 1;
00665 }
```

2.1.2.67 obj_clear()

```
static SQInteger obj_clear (
    HSQUIRRELVM v ) [static]
```

コンテナ（テーブルや配列）の内容をクリアします。

clear() メソッドの実装。対象コンテナのすべての要素を削除します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 648 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
00649 {
00650     return SQ_SUCCEEDED(sq_clear(v,-1)) ? 1 : SQ_ERROR;
00651 }
```

2.1.2.68 obj_delegate_weakref()

```
static SQInteger obj_delegate_weakref (
    HSQUIRRELVM v ) [static]
```

オブジェクトへの弱い参照 (weak reference) を作成します。

weakref() メソッドの実装。ガベージコレクタがオブジェクトを回収することを妨げない参照を作成します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 1。

Definition at line 636 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
00637 {
00638     sq_weakref(v, 1);
00639     return 1;
00640 }
```

2.1.2.69 sq_base_register()

```
void sq_base_register (
    HSQUIRRELVm v )
```

ベースライブラリをVMに登録します。

base_funcs 配列に定義された全てのグローバル関数と、バージョン情報などの定数をVMのルートテーブルに登録します。

Parameters

v	登録対象のSquirrel VM のハンドル。
---	-------------------------

Definition at line 507 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
00508 {
00509     SQInteger i=0;
00510     sq_pushroottable(v);
00511     while(base_funcs[i].name!=0) {
00512         sq_pushstring(v, base_funcs[i].name, -1);
00513         sq_newclosure(v, base_funcs[i].f, 0);
00514         sq_setnativeclosurename(v, -1, base_funcs[i].name);
00515         sq_setparamscheck(v, base_funcs[i].nparamscheck, base_funcs[i].typemask);
00516         sq_newslot(v, -3, SQFalse);
00517         i++;
00518     }
00519
00520     sq_pushstring(v, _SC("_versionnumber_"), -1);
00521     sq_pushinteger(v, SQUIRREL_VERSION_NUMBER);
00522     sq_newslot(v, -3, SQFalse);
00523     sq_pushstring(v, _SC("_version_"), -1);
00524     sq_pushstring(v, SQUIRREL_VERSION, -1);
00525     sq_newslot(v, -3, SQFalse);
00526     sq_pushstring(v, _SC("_charsize_"), -1);
00527     sq_pushinteger(v, sizeof(SQChar));
00528     sq_newslot(v, -3, SQFalse);
00529     sq_pushstring(v, _SC("_intsize_"), -1);
00530     sq_pushinteger(v, sizeof(SQInteger));
00531     sq_newslot(v, -3, SQFalse);
00532     sq_pushstring(v, _SC("_floatsize_"), -1);
00533     sq_pushinteger(v, sizeof(SQFloat));
00534     sq_newslot(v, -3, SQFalse);
00535     sq_pop(v, 1);
00536 }
```

References [base_funcs](#).

2.1.2.70 str2num()

```
static bool str2num (
```

```
const SQChar * s,
SQObjectPtr & res,
SQInteger base ) [static]
```

文字列を数値（浮動小数点数または整数）に変換します。

文字列を解析し、浮動小数点数または指定された基数の整数に変換します。文字列に '!' や 'e'/'E' (科学技術表記) が含まれる場合は浮動小数点数として、そうでない場合は整数として変換を試みます。13 進数より大きい基数の場合、'e' は指数ではなく 16 進数の文字として扱われます。

Parameters

in	s	変換対象のNULL 終端文字列。
out	res	変換された数値を格納するSQObjectPtr。
in	base	整数に変換する際の基数（2 から 36）。

Returns

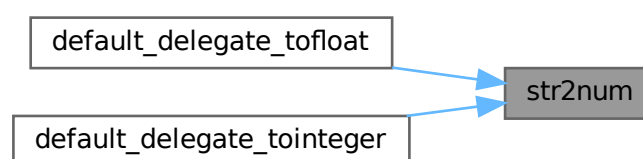
変換に成功した場合は `true`、失敗した場合は `false` を返します。

Definition at line 31 of file `squirrel3_squirrel_sqbaselib_v1.cpp`.

```
00032 {
00033     SQChar *end;
00034     const SQChar *e = s;
00035     bool iseintbase = base > 13; //to fix error converting hexadecimals with e like 56f0791e
00036     bool isfloat = false;
00037     SQChar c;
00038     while((c = *e) != _SC('\0'))
00039     {
00040         if (c == _SC('.') || (!iseintbase && (c == _SC('E') || c == _SC('e')))) { //e and E is for
scientific notation
00041             isfloat = true;
00042             break;
00043         }
00044         e++;
00045     }
00046     if(isfloat){
00047         SQFloat r = SQFloat(scstrtod(s,&end));
00048         if(s == end) return false;
00049         res = r;
00050     }
00051     else{
00052         SQInteger r = SQInteger(scstrtol(s,&end,(int)base));
00053         if(s == end) return false;
00054         res = r;
00055     }
00056     return true;
00057 }
```

Referenced by [default_delegate_tofloat\(\)](#), and [default_delegate_tointeger\(\)](#).

Here is the caller graph for this function:



2.1.2.71 string_find()

```
static SQInteger string_find (
    HSQUIRRELVM v ) [static]
```

文字列内から部分文字列を検索し、最初のインデックスを返します。

find() メソッドの実装。オプションで検索開始位置を指定できます。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1（インデックスをスタックにプッシュ）。見つからない場合は 0。

Definition at line 1404 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01405 {
01406     SQInteger top,start_idx=0;
01407     const SQChar *str,*substr,*ret;
01408     if(((top=sq_gettop(v))>1) && SQ_SUCCEEDED(sq_getstring(v,1,&str)) &&
        SQ_SUCCEEDED(sq_getstring(v,2,&substr))) {
01409         if(top>2) sq_getinteger(v,3,&start_idx);
01410         if((sq_getsize(v,1)>start_idx) && (start_idx>=0)) {
01411             ret=scstrstr(&str[start_idx],substr);
01412             if(ret){
01413                 sq_pushinteger(v,(SQInteger)(ret-str));
01414                 return 1;
01415             }
01416         }
01417         return 0;
01418     }
01419     return sq_throwerror(v,_SC("invalid param"));
01420 }
```

2.1.2.72 string_slice()

```
static SQInteger string_slice (
    HSQUIRRELVM v ) [static]
```

文字列の一部を切り出した新しい文字列（スライス）を返します。

slice() メソッドの実装。開始インデックスと終了インデックスを指定します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 1。結果の文字列をスタックにプッシュします。インデックスが不正な場合はエラー。

Definition at line 1384 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```
01385 {
01386     SQInteger sidx,eidx;
01387     SQObjectPtr o;
01388     if(SQ_FAILED(get_slice_params(v,sidx,eidx,o))) return -1;
```

```

01389     SQInteger slen = _string(o)->_len;
01390     if(sidx < 0)sidx = slen + sidx;
01391     if(eidx < 0)eidx = slen + eidx;
01392     if(eidx < sidx) return sq_throwerror(v,_SC("wrong indexes"));
01393     if(eidx > slen || sidx < 0) return sq_throwerror(v,_SC("slice out of range"));
01394     v->Push(SQString::Create(_ss(v),&_stringval(o)[sidx],eidx-sidx));
01395     return 1;
01396 }

```

References [get_slice_params\(\)](#).

Here is the call graph for this function:



2.1.2.73 table_filter()

```

static SQInteger table_filter (
    HSQUIRRELVM v ) [static]

```

テーブルの要素をフィルタリングします。

`filter()` メソッドの実装。各キーと値のペアに対して述語関数を呼び出し、その結果が `true` であったペアのみを含む新しいテーブルを生成します。

Parameters

<code>v</code>	Squirrel VM のハンドル。
----------------	--------------------

Returns

常に 1。結果のテーブルをスタックにプッシュします。エラー発生時は `SQ_ERROR`。

Definition at line 755 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

00756 {
00757     SQObject &o = stack_get(v,1);
00758     SQTable *tbl = _table(o);
00759     SQObjectPtr ret = SQTable::Create(_ss(v),0);
00760
00761     SQObjectPtr itr, key, val;
00762     SQInteger nitr;
00763     while((nitr = tbl->Next(false, itr, key, val)) != -1) {
00764         itr = (SQInteger)nitr;
00765
00766         v->Push(o);
00767         v->Push(key);
00768         v->Push(val);
00769         if(SQ_FAILED(sq_call(v,3,SQTrue,SQFalse))) {
00770             return SQ_ERROR;
00771         }
00772         if(!SQVM::IsFalse(v->GetUp(-1))) {
00773             _table(ret)->NewSlot(key, val);
00774         }
00775         v->Pop();

```

```

00776     }
00777
00778     v->Push(ret);
00779     return 1;
00780 }
```

2.1.2.74 table_getdelegate()

```

static SQInteger table_getdelegate (
    HSQUIRRELVM v ) [static]
```

テーブルのデリゲートを取得します。

getdelegate() メソッドの実装。テーブルの現在のデリゲートを取得し、スタックにプッシュします。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 743 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

00744 {
00745     return SQ_SUCCEEDED(sq_getdelegate(v, -1)) ? 1 : SQ_ERROR;
00746 }
```

2.1.2.75 table_map()

```

static SQInteger table_map (
    HSQUIRRELVM v ) [static]
```

テーブルの各要素を変換（マップ）します。

map() メソッドの実装。テーブルの各キーと値のペアに対して関数を呼び出し、その戻り値からなる新しい配列を生成します。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

常に 1。結果の配列をスタックにプッシュします。エラー発生時はSQ_ERROR。

Definition at line 789 of file [squirrel3_squirrel_sqbaselib_v1.cpp](#).

```

00790 {
00791     SQObject &o = stack_get(v, 1);
00792     SQTable *tbl = _table(o);
00793     SQInteger nitr, n = 0;
00794     SQInteger nitems = tbl->CountUsed();
00795     SQObjectPtr ret = SQArray::Create(_ss(v), nitems);
00796     SQObjectPtr itr, key, val;
00797     while ((nitr = tbl->Next(false, itr, key, val)) != -1) {
```

```

00798         itr = (SQInteger)nitr;
00799
00800         v->Push(o);
00801         v->Push(key);
00802         v->Push(val);
00803         if (SQ_FAILED(sq_call(v, 3, SQTrue, SQFalse))) {
00804             return SQ_ERROR;
00805         }
00806         _array(ret)->Set(n, v->GetUp(-1));
00807         v->Pop();
00808         n++;
00809     }
00810
00811     v->Push(ret);
00812     return 1;
00813 }

```

2.1.2.76 table_rawdelete()

```

static SQInteger table_rawdelete (
    HSQUIRRELVm v ) [static]

```

テーブルからキーと値のペアをデリゲートを無視して削除します。

rawdelete() メソッドの実装。_delslot メタメソッドをトリガーせずにスロットを直接削除します。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 678 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```

00679 {
00680     if (SQ_FAILED(sq_rawdeleteslot(v,1,SQTrue)))
00681         return SQ_ERROR;
00682     return 1;
00683 }

```

2.1.2.77 table_setdelegate()

```

static SQInteger table_setdelegate (
    HSQUIRRELVm v ) [static]

```

テーブルにデリゲートを設定します。

setdelegate() メソッドの実装。テーブルのデリゲート（プロトタイプ）を設定します。

Parameters

<i>v</i>	Squirrel VM のハンドル。
----------	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 729 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
00730 {
00731     if (SQ_FAILED(sq_setdelegate(v, -2)))
00732         return SQ_ERROR;
00733     sq_push(v, -1); // -1 because sq_setdelegate pops 1
00734     return 1;
00735 }
```

2.1.2.78 thread_call()

```
static SQInteger thread_call (
    HSQUIRRELVN v ) [static]
```

スレッドを開始または再開します。

call() メソッドの実装。スレッドのルートクロージャを呼び出します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1692 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
01693 {
01694     SQObjectPtr o = stack_get(v, 1);
01695     if (sq_type(o) == OT_THREAD) {
01696         SQInteger nparams = sq_gettop(v);
01697         _thread(o)->Push(_thread(o)->_roottable);
01698         for (SQInteger i = 2; i < (nparams+1); i++)
01699             sq_move(_thread(o), v, i);
01700         if (SQ_SUCCEEDED(sq_call(_thread(o), nparams, SQTrue, SQTrue))) {
01701             sq_move(v, _thread(o), -1);
01702             sq_pop(_thread(o), 1);
01703             return 1;
01704         }
01705         v->_lasterror = _thread(o)->_lasterror;
01706         return SQ_ERROR;
01707     }
01708     return sq_throwerror(v, _SC("wrong parameter"));
01709 }
```

2.1.2.79 thread_getstackinfos()

```
static SQInteger thread_getstackinfos (
    HSQUIRRELVN v ) [static]
```

指定されたスレッドのコールスタック情報を取得します。

getstackinfos() メソッドの実装。対象スレッドのスタック情報をテーブルとして返します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR または 0。

Definition at line 1831 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```

01832 {
01833     SQObjectPtr o = stack_get(v,1);
01834     if(sq_type(o) == OT_THREAD) {
01835         SQVM *thread = _thread(o);
01836         SQInteger threadtop = sq_gettop(thread);
01837         SQInteger level;
01838         sq_getinteger(v,-1,&level);
01839         SQRESULT res = __getcallstackinfos(thread,level);
01840         if(SQ_FAILED(res))
01841         {
01842             sq_settop(thread,threadtop);
01843             if(sq_type(thread->_lasterror) == OT_STRING) {
01844                 sq_throwerror(v,_stringval(thread->_lasterror));
01845             }
01846             else {
01847                 sq_throwerror(v,_SC("unknown error"));
01848             }
01849         }
01850         if(res > 0) {
01851             //some result
01852             sq_move(v,thread,-1);
01853             sq_settop(thread,threadtop);
01854             return 1;
01855         }
01856         //no result
01857         sq_settop(thread,threadtop);
01858         return 0;
01859     }
01860 }
01861 return sq_throwerror(v,_SC("wrong parameter"));
01862 }
```

References [__getcallstackinfos\(\)](#).

Here is the call graph for this function:



2.1.2.80 thread_getstatus()

```
static SQInteger thread_getstatus (
    HSQUIRRELVm v ) [static]
```

スレッドの現在の状態を取得します。

getstatus() メソッドの実装。状態は "idle", "running", "suspended" のいずれかです。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

常に 1。状態を表す文字列をスタックにプッシュします。

Definition at line 1806 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```

01807 {
01808     SQObjectPtr &o = stack_get(v,1);
01809     switch(sq_getvmstate(_thread(o))) {
01810         case SQ_VMSTATE_IDLE:
01811             sq_pushstring(v,_SC("idle"),-1);
01812             break;
01813         case SQ_VMSTATE_RUNNING:
01814             sq_pushstring(v,_SC("running"),-1);
01815             break;
01816         case SQ_VMSTATE_SUSPENDED:
01817             sq_pushstring(v,_SC("suspended"),-1);
01818             break;
01819         default:
01820             return sq_throwerror(v,_SC("internal VM error"));
01821     }
01822     return 1;
01823 }
```

2.1.2.81 thread_wakeup()

```
static SQInteger thread_wakeup (
    HSQUIRRELVM v ) [static]
```

中断状態のスレッドを再開します。

wakeup() メソッドの実装。オプションで値をスレッドに渡すことができます。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合は SQ_ERROR。

Definition at line 1717 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```

01718 {
01719     SQObjectPtr o = stack_get(v,1);
01720     if(sq_type(o) == OT_THREAD) {
01721         SQVM *thread = _thread(o);
01722         SQInteger state = sq_getvmstate(thread);
01723         if(state != SQ_VMSTATE_SUSPENDED) {
01724             switch(state) {
01725                 case SQ_VMSTATE_IDLE:
01726                     return sq_throwerror(v,_SC("cannot wakeup a idle thread"));
01727                 case SQ_VMSTATE_RUNNING:
01728                     return sq_throwerror(v,_SC("cannot wakeup a running thread"));
01729                 break;
01730             }
01731         }
01732     }
01733     SQInteger wakeupret = sq_gettop(v)>1?SQTrue:SQFalse;
01734     if(wakeupret) {
01735         sq_move(thread,v,2);
01736     }
01737     if(SQ_SUCCEEDED(sq_wakeupvm(thread,wakeupret,SQTrue,SQTrue,SQFalse))) {
01738         sq_move(v,thread,-1);
01739         sq_pop(thread,1); //pop retval
01740         if(sq_getvmstate(thread) == SQ_VMSTATE_IDLE) {
01741             sq_settop(thread,1); //pop roottable
01742         }
01743         return 1;
01744     }
01745 }
```

```

01746         sq_settop(thread,1);
01747         v->_lasterror = thread->_lasterror;
01748         return SQ_ERROR;
01749     }
01750     return sq_throwerror(v,_SC("wrong parameter"));
01751 }

```

2.1.2.82 thread_wakeupthrow()

```

static SQInteger thread_wakeupthrow (
    HSQUIRRELVm v ) [static]

```

中断状態のスレッドに例外をスローして再開させます。

wakeupthrow() メソッドの実装。指定されたオブジェクトをスレッド内でスローして実行を再開します。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 1759 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```

01760 {
01761     SQObjectPtr o = stack_get(v,1);
01762     if(sq_type(o) == OT_THREAD) {
01763         SQVM *thread = _thread(o);
01764         SQInteger state = sq_getvmstate(thread);
01765         if(state != SQ_VMSTATE_SUSPENDED) {
01766             switch(state) {
01767                 case SQ_VMSTATE_IDLE:
01768                     return sq_throwerror(v,_SC("cannot wakeup a idle thread"));
01769                 break;
01770                 case SQ_VMSTATE_RUNNING:
01771                     return sq_throwerror(v,_SC("cannot wakeup a running thread"));
01772                 break;
01773             }
01774         }
01775
01776         sq_move(thread,v,2);
01777         sq_throwobject(thread);
01778         SQBool rethrow_error = SQTrue;
01779         if(sq_gettop(v) > 2) {
01780             sq_getbool(v,3,&rethrow_error);
01781         }
01782         if(SQ_SUCCEEDED(sq_wakeupvm(thread,SQFalse,SQTrue,SQTrue,SQTrue))) {
01783             sq_move(v,thread,-1);
01784             sq_pop(thread,1); //pop retval
01785             if(sq_getvmstate(thread) == SQ_VMSTATE_IDLE) {
01786                 sq_settop(thread,1); //pop roottable
01787             }
01788             return 1;
01789         }
01790         sq_settop(thread,1);
01791         if(rethrow_error) {
01792             v->_lasterror = thread->_lasterror;
01793             return SQ_ERROR;
01794         }
01795         return SQ_OK;
01796     }
01797     return sq_throwerror(v,_SC("wrong parameter"));
01798 }

```

2.1.2.83 weakref_ref()

```

static SQInteger weakref_ref (
    HSQUIRRELVm v ) [static]

```


弱い参照 (weak reference) から元のオブジェクトを取得します。

ref() メソッドの実装。弱い参照が指すオブジェクトをスタックにプッシュします。オブジェクトが既に解放済みの場合は null をプッシュします。

Parameters

v	Squirrel VM のハンドル。
---	--------------------

Returns

成功した場合は 1、失敗した場合はSQ_ERROR。

Definition at line 2016 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
02017 {
02018     if(SQ_FAILED(sq_getweakrefval(v,1)))
02019         return SQ_ERROR;
02020     return 1;
02021 }
```

2.1.3 Variable Documentation

2.1.3.1 base_funcs

```
const SQRegFunction base_funcs[] [static]
```

Initial value:

```
={
    {_SC("seterrorhandler"),base_seterrorhandler,2, NULL},
    {_SC("setdebughook"),base_setdebughook,2, NULL},
    {_SC("enabledebuginfo"),base_enabledebuginfo,2, NULL},
    {_SC("getstackinfos"),base_getstackinfos,2, _SC(".n")},
    {_SC("getroottable"),base_getroottable,1, NULL},
    {_SC("setroottable"),base_setroottable,2, NULL},
    {_SC("getconsttable"),base_getconsttable,1, NULL},
    {_SC("setconsttable"),base_setconsttable,2, NULL},
    {_SC("assert"),base_assert,-2, NULL},
    {_SC("print"),base_print,2, NULL},
    {_SC("error"),base_error,2, NULL},
    {_SC("compilestring"),base_compilestring,-2, _SC(".ss")},
    {_SC("newthread"),base_newthread,2, _SC(".c")},
    {_SC("suspend"),base_suspend,-1, NULL},
    {_SC("array"),base_array,-2, _SC(".n")},
    {_SC("type"),base_type,2, NULL},
    {_SC("callee"),base_callee,0,NULL},
    {_SC("dummy"),base_dummy,0,NULL},

    {_SC("collectgarbage"),base_collectgarbage,0, NULL},
    {_SC("resurrectunreachable"),base_resurrectunreachable,0, NULL},

    {NULL, (SQFUNCTION)0,0,NULL}
}
```

Squirrel のベースライブラリ関数を定義する静的配列。

Definition at line 473 of file squirrel3_squirrel_sqbaselib_v1.cpp.

```
00473 {
00474     //generic
00475     {_SC("seterrorhandler"),base_seterrorhandler,2, NULL},
00476     {_SC("setdebughook"),base_setdebughook,2, NULL},
00477     {_SC("enabledebuginfo"),base_enabledebuginfo,2, NULL},
00478     {_SC("getstackinfos"),base_getstackinfos,2, _SC(".n")},
00479     {_SC("getroottable"),base_getroottable,1, NULL},
00480     {_SC("setroottable"),base_setroottable,2, NULL},
00481     {_SC("getconsttable"),base_getconsttable,1, NULL},
00482     {_SC("setconsttable"),base_setconsttable,2, NULL},
00483     {_SC("assert"),base_assert,-2, NULL},
```

```

00484     {_SC("print"),base_print,2, NULL},
00485     {_SC("error"),base_error,2, NULL},
00486     {_SC("compilestring"),base_compilestring,-2, _SC(".ss")},
00487     {_SC("newthread"),base_newthread,2, _SC(".c")},
00488     {_SC("suspend"),base_suspend,-1, NULL},
00489     {_SC("array"),base_array,-2, _SC(".n")},
00490     {_SC("type"),base_type,2, NULL},
00491     {_SC("callee"),base_callee,0,NULL},
00492     {_SC("dummy"),base_dummy,0,NULL},
00493 #ifndef NO_GARBAGE_COLLECTOR
00494     {_SC("collectgarbage"),base_collectgarbage,0, NULL},
00495     {_SC("resurrectunreachable"),base_resurrectureachable,0, NULL},
00496 #endif
00497     {NULL, (SQFUNCTION) 0, 0, NULL}
00498 };

```

Referenced by [sq_base_register\(\)](#).

2.2 squirrel3_squirrel_sqbaselib_v1.cpp

[Go to the documentation of this file.](#)

```

00001 //cpp
00002
00003 /*
00004     Written By Gemini
00005 */
00006
00007 #include "sqpheader.h"
00008 #include "sqvm.h"
00009 #include "sqstring.h"
00010 #include "sqtable.h"
00011 #include "sqarray.h"
00012 #include "sqfuncproto.h"
00013 #include "sqclosure.h"
00014 #include "sqclass.h"
00015 #include <stdlib.h>
00016 #include <stdarg.h>
00017 #include <ctype.h>
00018
00031 static bool str2num(const SQChar *s, SQObjectPtr &res, SQInteger base)
00032 {
00033     SQChar *end;
00034     const SQChar *e = s;
00035     bool isintbase = base > 13; //to fix error converting hexadecimals with e like 56f0791e
00036     bool isfloat = false;
00037     SQChar c;
00038     while((c = *e) != _SC('\0'))
00039     {
00040         if (c == _SC('.') || (!isintbase && (c == _SC('E') || c == _SC('e')))) { //e and E is for
scientific notation
00041             isfloat = true;
00042             break;
00043         }
00044         e++;
00045     }
00046     if(isfloat){
00047         SQFloat r = SQFloat(scstrtod(s,&end));
00048         if(s == end) return false;
00049         res = r;
00050     }
00051     else{
00052         SQInteger r = SQInteger(scstrtol(s,&end,(int)base));
00053         if(s == end) return false;
00054         res = r;
00055     }
00056     return true;
00057 }
00058
00067 static SQInteger base_dummy(HSQUIRRELVLM SQ_UNUSED_ARG(v))
00068 {
00069     return 0;
00070 }
00071
00072 #ifndef NO_GARBAGE_COLLECTOR
00081 static SQInteger base_collectgarbage(HSQUIRRELVLM v)
00082 {
00083     sq_pushinteger(v, sq_collectgarbage(v));
00084     return 1;
00085 }
00094 static SQInteger base_resurrectureachable(HSQUIRRELVLM v)

```

```

00095 {
00096     sq_resurrectunreachable(v);
00097     return 1;
00098 }
00099 #endif
00100
00107 static SQInteger base_getroottable(HSQIRRELVM v)
00108 {
00109     v->Push(v->_roottable);
00110     return 1;
00111 }
00112
00119 static SQInteger base_getconsttable(HSQIRRELVM v)
00120 {
00121     v->Push(_ss(v)->_consts);
00122     return 1;
00123 }
00124
00133 static SQInteger base_setroottable(HSQIRRELVM v)
00134 {
00135     SQObjectPtr o = v->_roottable;
00136     if(SQ_FAILED(sq_setroottable(v))) return SQ_ERROR;
00137     v->Push(o);
00138     return 1;
00139 }
00140
00149 static SQInteger base_setconsttable(HSQIRRELVM v)
00150 {
00151     SQObjectPtr o = _ss(v)->_consts;
00152     if(SQ_FAILED(sq_setconsttable(v))) return SQ_ERROR;
00153     v->Push(o);
00154     return 1;
00155 }
00156
00163 static SQInteger base_seterrorhandler(HSQIRRELVM v)
00164 {
00165     sq_seterrorhandler(v);
00166     return 0;
00167 }
00168
00175 static SQInteger base_setdebughook(HSQIRRELVM v)
00176 {
00177     sq_setdebughook(v);
00178     return 0;
00179 }
00180
00189 static SQInteger base_enableddebuginfo(HSQIRRELVM v)
00190 {
00191     SQObjectPtr &o=stack_get(v,2);
00192
00193     sq_enableddebuginfo(v,SQVM::IsFalse(o)?SQFalse:SQTrue);
00194     return 0;
00195 }
00196
00206 static SQInteger __getcallstackinfos(HSQIRRELVM v,SQInteger level)
00207 {
00208     SQStackInfos si;
00209     SQInteger seq = 0;
00210     const SQChar *name = NULL;
00211
00212     if (SQ_SUCCEEDED(sq_stackinfos(v, level, &si))
00213     {
00214         const SQChar *fn = _SC("unknown");
00215         const SQChar *src = _SC("unknown");
00216         if(si.funcname) fn = si.funcname;
00217         if(si.source) src = si.source;
00218         sq_newtable(v);
00219         sq_pushstring(v, _SC("func"), -1);
00220         sq_pushstring(v, fn, -1);
00221         sq_newslot(v, -3, SQFalse);
00222         sq_pushstring(v, _SC("src"), -1);
00223         sq_pushstring(v, src, -1);
00224         sq_newslot(v, -3, SQFalse);
00225         sq_pushstring(v, _SC("line"), -1);
00226         sq_pushinteger(v, si.line);
00227         sq_newslot(v, -3, SQFalse);
00228         sq_pushstring(v, _SC("locals"), -1);
00229         sq_newtable(v);
00230         seq=0;
00231         while ((name = sq_getlocal(v, level, seq)) {
00232             sq_pushstring(v, name, -1);
00233             sq_push(v, -2);
00234             sq_newslot(v, -4, SQFalse);
00235             sq_pop(v, 1);
00236             seq++;
00237         }
00238         sq_newslot(v, -3, SQFalse);

```

```

00239         return 1;
00240     }
00241
00242     return 0;
00243 }
00244
00253 static SQInteger base_getstackinfos(HSQIRRELVM v)
00254 {
00255     SQInteger level;
00256     sq_getinteger(v, -1, &level);
00257     return __getcallstackinfos(v, level);
00258 }
00259
00268 static SQInteger base_assert(HSQIRRELVM v)
00269 {
00270     if(SQVM::IsFalse(stack_get(v, 2))) {
00271         SQInteger top = sq_gettop(v);
00272         if (top>2 && SQ_SUCCEEDED(sq_tostring(v, 3))) {
00273             const SQChar *str = 0;
00274             if (SQ_SUCCEEDED(sq_getstring(v, -1, &str))) {
00275                 return sq_throwerror(v, str);
00276             }
00277         }
00278         return sq_throwerror(v, _SC("assertion failed"));
00279     }
00280     return 0;
00281 }
00282
00294 static SQInteger get_slice_params(HSQIRRELVM v, SQInteger &sidx, SQInteger &eidx, SQObjectPtr &o)
00295 {
00296     SQInteger top = sq_gettop(v);
00297     sidx=0;
00298     eidx=0;
00299     o=stack_get(v, 1);
00300     if(top>1){
00301         SQObjectPtr &start=stack_get(v, 2);
00302         if(sq_type(start)!=OT_NULL && sq_isnumeric(start)){
00303             sidx=tointeger(start);
00304         }
00305     }
00306     if(top>2){
00307         SQObjectPtr &end=stack_get(v, 3);
00308         if(sq_isnumeric(end)){
00309             eidx=tointeger(end);
00310         }
00311     }
00312     else {
00313         eidx = sq_getsize(v, 1);
00314     }
00315     return 1;
00316 }
00317
00326 static SQInteger base_print(HSQIRRELVM v)
00327 {
00328     const SQChar *str;
00329     if(SQ_SUCCEEDED(sq_tostring(v, 2)))
00330     {
00331         if(SQ_SUCCEEDED(sq_getstring(v, -1, &str))) {
00332             if(_ss(v)->_printfunc) _ss(v)->_printfunc(v, _SC("%s"), str);
00333             return 0;
00334         }
00335     }
00336     return SQ_ERROR;
00337 }
00338
00347 static SQInteger base_error(HSQIRRELVM v)
00348 {
00349     const SQChar *str;
00350     if(SQ_SUCCEEDED(sq_tostring(v, 2)))
00351     {
00352         if(SQ_SUCCEEDED(sq_getstring(v, -1, &str))) {
00353             if(_ss(v)->_errorfunc) _ss(v)->_errorfunc(v, _SC("%s"), str);
00354             return 0;
00355         }
00356     }
00357     return SQ_ERROR;
00358 }
00359
00368 static SQInteger base_compilestring(HSQIRRELVM v)
00369 {
00370     SQInteger nargs=sq_gettop(v);
00371     const SQChar *src=NULL, *name=_SC("unnamedbuffer");
00372     SQInteger size;
00373     sq_getstring(v, 2, &src);
00374     size=sq_getsize(v, 2);
00375     if(nargs>2){
00376         sq_getstring(v, 3, &name);

```

```

00377     }
00378     if(SQ_SUCCEEDED(sq_compilebuffer(v,src,size,name,SQFalse)))
00379         return 1;
00380     else
00381         return SQ_ERROR;
00382 }
00383
00392 static SQInteger base_newthread(HSQIRRELVM v)
00393 {
00394     SQObjectPtr &func = stack_get(v,2);
00395     SQInteger stksize = (_closure(func)->_function->_stacksize << 1) +2;
00396     HSQIRRELVM newv = sq_newthread(v, (stksize < MIN_STACK_OVERHEAD + 2)? MIN_STACK_OVERHEAD + 2 :
stksize);
00397     sq_move(newv,v,-2);
00398     return 1;
00399 }
00400
00409 static SQInteger base_suspend(HSQIRRELVM v)
00410 {
00411     return sq_suspendvm(v);
00412 }
00413
00422 static SQInteger base_array(HSQIRRELVM v)
00423 {
00424     SQArray *a;
00425     SQObject &size = stack_get(v,2);
00426     if(sq_gettop(v) > 2) {
00427         a = SQArray::Create(_ss(v),0);
00428         a->Resize(tointeger(size),stack_get(v,3));
00429     }
00430     else {
00431         a = SQArray::Create(_ss(v),tointeger(size));
00432     }
00433     v->Push(a);
00434     return 1;
00435 }
00436
00445 static SQInteger base_type(HSQIRRELVM v)
00446 {
00447     SQObjectPtr &o = stack_get(v,2);
00448     v->Push(SQString::Create(_ss(v),GetTypeName(o),-1));
00449     return 1;
00450 }
00451
00460 static SQInteger base_callee(HSQIRRELVM v)
00461 {
00462     if(v->_callsstacksize > 1)
00463     {
00464         v->Push(v->_callsstack[v->_callsstacksize - 2]._closure);
00465         return 1;
00466     }
00467     return sq_throwerror(v,_SC("no closure in the calls stack"));
00468 }
00469
00473 static const SQRegFunction base_funcs[]={
00474     //generic
00475     {_SC("seterrorhandler"),base_seterrorhandler,2, NULL},
00476     {_SC("setdebughook"),base_setdebughook,2, NULL},
00477     {_SC("enabledebuginfo"),base_enabledebuginfo,2, NULL},
00478     {_SC("getstackinfos"),base_getstackinfos,2, _SC(".n")},
00479     {_SC("getroottable"),base_getroottable,1, NULL},
00480     {_SC("setroottable"),base_setroottable,2, NULL},
00481     {_SC("getconsttable"),base_getconsttable,1, NULL},
00482     {_SC("setconsttable"),base_setconsttable,2, NULL},
00483     {_SC("assert"),base_assert,-2, NULL},
00484     {_SC("print"),base_print,2, NULL},
00485     {_SC("error"),base_error,2, NULL},
00486     {_SC("compilestring"),base_compilestring,-2, _SC(".ss")},
00487     {_SC("newthread"),base_newthread,2, _SC(".c")},
00488     {_SC("suspend"),base_suspend,-1, NULL},
00489     {_SC("array"),base_array,-2, _SC(".n")},
00490     {_SC("type"),base_type,2, NULL},
00491     {_SC("callee"),base_callee,0,NULL},
00492     {_SC("dummy"),base_dummy,0,NULL},
00493 #ifndef NO_GARBAGE_COLLECTOR
00494     {_SC("collectgarbage"),base_collectgarbage,0, NULL},
00495     {_SC("resurrectunreachable"),base_resurrectunreachable,0, NULL},
00496 #endif
00497     {NULL, (SQFUNCTION)0,0,NULL}
00498 };
00499
00507 void sq_base_register(HSQIRRELVM v)
00508 {
00509     SQInteger i=0;
00510     sq_pushroottable(v);
00511     while(base_funcs[i].name!=0) {
00512         sq_pushstring(v,base_funcs[i].name,-1);

```

```

00513         sq_newclosure(v, base_funcs[i].f, 0);
00514         sq_setnativeclosurename(v, -1, base_funcs[i].name);
00515         sq_setparamscheck(v, base_funcs[i].nparamscheck, base_funcs[i].typemask);
00516         sq_newslot(v, -3, SQFalse);
00517         i++;
00518     }
00519
00520     sq_pushstring(v, _SC("_versionnumber_"), -1);
00521     sq_pushinteger(v, SQUIRREL_VERSION_NUMBER);
00522     sq_newslot(v, -3, SQFalse);
00523     sq_pushstring(v, _SC("_version_"), -1);
00524     sq_pushstring(v, SQUIRREL_VERSION, -1);
00525     sq_newslot(v, -3, SQFalse);
00526     sq_pushstring(v, _SC("_charsize_"), -1);
00527     sq_pushinteger(v, sizeof(SQChar));
00528     sq_newslot(v, -3, SQFalse);
00529     sq_pushstring(v, _SC("_intsize_"), -1);
00530     sq_pushinteger(v, sizeof(SQInteger));
00531     sq_newslot(v, -3, SQFalse);
00532     sq_pushstring(v, _SC("_floatsize_"), -1);
00533     sq_pushinteger(v, sizeof(SQFloat));
00534     sq_newslot(v, -3, SQFalse);
00535     sq_pop(v, 1);
00536 }
00537
00544 static SQInteger default_delegate_len(HSQUIRRELVm v)
00545 {
00546     v->Push(SQInteger(sq_getsize(v, 1)));
00547     return 1;
00548 }
00549
00556 static SQInteger default_delegate_tofloat(HSQUIRRELVm v)
00557 {
00558     SQObjectPtr &o=stack_get(v, 1);
00559     switch(sq_type(o)) {
00560     case OT_STRING: {
00561         SQObjectPtr res;
00562         if(str2num(_stringval(o), res, 10)) {
00563             v->Push(SQObjectPtr(tofloat(res)));
00564             break;
00565         }
00566         return sq_throwerror(v, _SC("cannot convert the string"));
00567         break;
00568     case OT_INTEGER: case OT_FLOAT:
00569         v->Push(SQObjectPtr(tofloat(o)));
00570         break;
00571     case OT_BOOL:
00572         v->Push(SQObjectPtr((SQFloat)(_integer(o)?1:0)));
00573         break;
00574     default:
00575         v->PushNull();
00576         break;
00577     }
00578     return 1;
00579 }
00580
00588 static SQInteger default_delegate_tointeger(HSQUIRRELVm v)
00589 {
00590     SQObjectPtr &o=stack_get(v, 1);
00591     SQInteger base = 10;
00592     if(sq_gettop(v) > 1) {
00593         sq_getinteger(v, 2, &base);
00594     }
00595     switch(sq_type(o)) {
00596     case OT_STRING: {
00597         SQObjectPtr res;
00598         if(str2num(_stringval(o), res, base)) {
00599             v->Push(SQObjectPtr(tointeger(res)));
00600             break;
00601         }
00602         return sq_throwerror(v, _SC("cannot convert the string"));
00603         break;
00604     case OT_INTEGER: case OT_FLOAT:
00605         v->Push(SQObjectPtr(tointeger(o)));
00606         break;
00607     case OT_BOOL:
00608         v->Push(SQObjectPtr(_integer(o)?(SQInteger)1:(SQInteger)0));
00609         break;
00610     default:
00611         v->PushNull();
00612         break;
00613     }
00614     return 1;
00615 }
00616
00623 static SQInteger default_delegate_tostring(HSQUIRRELVm v)
00624 {

```

```

00625     if (SQ_FAILED(sq_tostring(v,1)))
00626         return SQ_ERROR;
00627     return 1;
00628 }
00629
00636 static SQInteger obj_delegate_weakref(HSQIRRELV v)
00637 {
00638     sq_weakref(v,1);
00639     return 1;
00640 }
00641
00648 static SQInteger obj_clear(HSQIRRELV v)
00649 {
00650     return SQ_SUCCEEDED(sq_clear(v,-1)) ? 1 : SQ_ERROR;
00651 }
00652
00659 static SQInteger number_delegate_tochar(HSQIRRELV v)
00660 {
00661     SQObject &o=stack_get(v,1);
00662     SQChar c = (SQChar)tointeger(o);
00663     v->Push(SQString::Create(_ss(v),(const SQChar *)&c,1));
00664     return 1;
00665 }
00666
00667
00668
00670 //TABLE DEFAULT DELEGATE
00671
00678 static SQInteger table_rawdelete(HSQIRRELV v)
00679 {
00680     if (SQ_FAILED(sq_rawdeleteslot(v,1,SQTrue)))
00681         return SQ_ERROR;
00682     return 1;
00683 }
00684
00691 static SQInteger container_rawexists(HSQIRRELV v)
00692 {
00693     if (SQ_SUCCEEDED(sq_rawget(v,-2))) {
00694         sq_pushbool(v,SQTrue);
00695         return 1;
00696     }
00697     sq_pushbool(v,SQFalse);
00698     return 1;
00699 }
00700
00707 static SQInteger container_rawset(HSQIRRELV v)
00708 {
00709     return SQ_SUCCEEDED(sq_rawset(v,-3)) ? 1 : SQ_ERROR;
00710 }
00711
00718 static SQInteger container_rawget(HSQIRRELV v)
00719 {
00720     return SQ_SUCCEEDED(sq_rawget(v,-2)) ? 1 : SQ_ERROR;
00721 }
00722
00729 static SQInteger table_setdelegate(HSQIRRELV v)
00730 {
00731     if (SQ_FAILED(sq_setdelegate(v,-2)))
00732         return SQ_ERROR;
00733     sq_push(v,-1); // -1 because sq_setdelegate pops 1
00734     return 1;
00735 }
00736
00743 static SQInteger table_getdelegate(HSQIRRELV v)
00744 {
00745     return SQ_SUCCEEDED(sq_getdelegate(v,-1)) ? 1 : SQ_ERROR;
00746 }
00747
00755 static SQInteger table_filter(HSQIRRELV v)
00756 {
00757     SQObject &o = stack_get(v,1);
00758     SQTable *tbl = _table(o);
00759     SQObjectPtr ret = SQTable::Create(_ss(v),0);
00760
00761     SQObjectPtr itr, key, val;
00762     SQInteger nitr;
00763     while((nitr = tbl->Next(false, itr, key, val)) != -1) {
00764         itr = (SQInteger)nitr;
00765
00766         v->Push(o);
00767         v->Push(key);
00768         v->Push(val);
00769         if (SQ_FAILED(sq_call(v,3,SQTrue,SQFalse))) {
00770             return SQ_ERROR;
00771         }
00772         if (!SQVM::IsFalse(v->GetUp(-1))) {
00773             _table(ret)->NewSlot(key, val);

```

```

00774     }
00775     v->Pop();
00776 }
00777
00778 v->Push(ret);
00779 return 1;
00780 }
00781
00789 static SQInteger table_map(HSQIRRELVM v)
00790 {
00791     SQObject &o = stack_get(v, 1);
00792     SQTable *tbl = _table(o);
00793     SQInteger nitr, n = 0;
00794     SQInteger nitems = tbl->CountUsed();
00795     SQObjectPtr ret = SQArray::Create(_ss(v), nitems);
00796     SQObjectPtr itr, key, val;
00797     while ((nitr = tbl->Next(false, itr, key, val)) != -1) {
00798         itr = (SQInteger)nitr;
00799
00800         v->Push(o);
00801         v->Push(key);
00802         v->Push(val);
00803         if (SQ_FAILED(sq_call(v, 3, SQTrue, SQFalse))) {
00804             return SQ_ERROR;
00805         }
00806         _array(ret)->Set(n, v->GetUp(-1));
00807         v->Pop();
00808         n++;
00809     }
00810
00811     v->Push(ret);
00812     return 1;
00813 }
00814
00820 #define TABLE_TO_ARRAY_FUNC(_funcname_,_valname_) static SQInteger _funcname_(HSQIRRELVM v) \
00821 { \
00822     SQObject &o = stack_get(v, 1); \
00823     SQTable *t = _table(o); \
00824     SQObjectPtr itr, key, val; \
00825     SQObjectPtr _null; \
00826     SQInteger nitr, n = 0; \
00827     SQInteger nitems = t->CountUsed(); \
00828     SQArray *a = SQArray::Create(_ss(v), nitems); \
00829     a->Resize(nitems, _null); \
00830     if (nitems) { \
00831         while ((nitr = t->Next(false, itr, key, val)) != -1) { \
00832             itr = (SQInteger)nitr; \
00833             a->Set(n, _valname_); \
00834             n++; \
00835         } \
00836     } \
00837     v->Push(a); \
00838     return 1; \
00839 }
00840
00845 TABLE_TO_ARRAY_FUNC(table_keys, key)
00850 TABLE_TO_ARRAY_FUNC(table_values, val)
00851
00852
00853
00856 const SQRegFunction SQSharedState::_table_default_delegate_funcz[]={
00857     {_SC("len"),default_delegate_len,1, _SC("t")},
00858     {_SC("rawget"),container_rawget,2, _SC("t")},
00859     {_SC("rawset"),container_rawset,3, _SC("t")},
00860     {_SC("rawdelete"),table_rawdelete,2, _SC("t")},
00861     {_SC("rawin"),container_rawexists,2, _SC("t")},
00862     {_SC("weakref"),obj_delegate_weakref,1, NULL },
00863     {_SC("tostring"),default_delegate_tostring,1, _SC(".")},
00864     {_SC("clear"),obj_clear,1, _SC(".")},
00865     {_SC("setdelegate"),table_setdelegate,2, _SC("t|o")},
00866     {_SC("getdelegate"),table_getdelegate,1, _SC(".")},
00867     {_SC("filter"),table_filter,2, _SC("tc")},
00868     {_SC("map"),table_map,2, _SC("tc") },
00869     {_SC("keys"),table_keys,1, _SC("t") },
00870     {_SC("values"),table_values,1, _SC("t") },
00871     {NULL, (SQFUNCTION)0,0,NULL}
00872 };
00873
00874 //ARRAY DEFAULT DELEGATE////////////////////////////////////
00875
00882 static SQInteger array_append(HSQIRRELVM v)
00883 {
00884     return SQ_SUCCEEDED(sq_arrayappend(v,-2)) ? 1 : SQ_ERROR;
00885 }
00886
00893 static SQInteger array_extend(HSQIRRELVM v)
00894 {

```



```

00895     _array(stack_get(v,1))->Extend(_array(stack_get(v,2)));
00896     sq_pop(v,1);
00897     return 1;
00898 }
00899
00906 static SQInteger array_reverse(HSQUIRRELV v)
00907 {
00908     return SQ_SUCCEEDED(sq_arrayreverse(v,-1)) ? 1 : SQ_ERROR;
00909 }
00910
00917 static SQInteger array_pop(HSQUIRRELV v)
00918 {
00919     return SQ_SUCCEEDED(sq_arraypop(v,1,SQTrue)) ? 1 : SQ_ERROR;
00920 }
00921
00928 static SQInteger array_top(HSQUIRRELV v)
00929 {
00930     SQObject &o=stack_get(v,1);
00931     if(_array(o)->Size()>0){
00932         v->Push(_array(o)->Top());
00933         return 1;
00934     }
00935     else return sq_throwerror(v,_SC("top() on a empty array"));
00936 }
00937
00944 static SQInteger array_insert(HSQUIRRELV v)
00945 {
00946     SQObject &o=stack_get(v,1);
00947     SQObject &idx=stack_get(v,2);
00948     SQObject &val=stack_get(v,3);
00949     if(!_array(o)->Insert(tointeger(idx),val))
00950         return sq_throwerror(v,_SC("index out of range"));
00951     sq_pop(v,2);
00952     return 1;
00953 }
00954
00961 static SQInteger array_remove(HSQUIRRELV v)
00962 {
00963     SQObject &o = stack_get(v, 1);
00964     SQObject &idx = stack_get(v, 2);
00965     if(!sq_isnumeric(idx)) return sq_throwerror(v, _SC("wrong type"));
00966     SQObjectPtr val;
00967     if(_array(o)->Get(tointeger(idx), val)) {
00968         _array(o)->Remove(tointeger(idx));
00969         v->Push(val);
00970         return 1;
00971     }
00972     return sq_throwerror(v, _SC("idx out of range"));
00973 }
00974
00981 static SQInteger array_resize(HSQUIRRELV v)
00982 {
00983     SQObject &o = stack_get(v, 1);
00984     SQObject &nsize = stack_get(v, 2);
00985     SQObjectPtr fill;
00986     if(sq_isnumeric(nsize)) {
00987         SQInteger sz = tointeger(nsize);
00988         if (sz<0)
00989             return sq_throwerror(v, _SC("resizing to negative length"));
00990
00991         if(sq_gettop(v) > 2)
00992             fill = stack_get(v, 3);
00993         _array(o)->Resize(sz,fill);
00994         sq_settop(v, 1);
00995         return 1;
00996     }
00997     return sq_throwerror(v, _SC("size must be a number"));
00998 }
00999
01008 static SQInteger __map_array(SQArray *dest,SQArray *src,HSQUIRRELV v) {
01009     SQObjectPtr temp;
01010     SQInteger size = src->Size();
01011     SQObject &closure = stack_get(v, 2);
01012     v->Push(closure);
01013
01014     SQInteger nArgs = 0;
01015     if(sq_type(closure) == OT_CLOSURE) {
01016         nArgs = _closure(closure)->_function->_nparameters;
01017     }
01018     else if (sq_type(closure) == OT_NATIVECLOSURE) {
01019         SQInteger nParamsCheck = _nativeclosure(closure)->_nparamscheck;
01020         if (nParamsCheck > 0)
01021             nArgs = nParamsCheck;
01022         else // push all params when there is no check or only minimal count set
01023             nArgs = 4;
01024     }
01025

```

```

01026     for(SQInteger n = 0; n < size; n++) {
01027         src->Get(n,temp);
01028         v->Push(src);
01029         v->Push(temp);
01030         if (nArgs >= 3)
01031             v->Push(SQObjectPtr(n));
01032         if (nArgs >= 4)
01033             v->Push(src);
01034         if (SQ_FAILED(sq_call(v,nArgs,SQTrue,SQFalse))) {
01035             return SQ_ERROR;
01036         }
01037         dest->Set(n,v->GetUp(-1));
01038         v->Pop();
01039     }
01040     v->Pop();
01041     return 0;
01042 }
01043
01050 static SQInteger array_map(HSQIRRELVM v)
01051 {
01052     SQObject &o = stack_get(v,1);
01053     SQInteger size = _array(o)->Size();
01054     SQObjectPtr ret = SQArray::Create(_ss(v),size);
01055     if (SQ_FAILED(_map_array(_array(ret),_array(o),v)))
01056         return SQ_ERROR;
01057     v->Push(ret);
01058     return 1;
01059 }
01060
01067 static SQInteger array_apply(HSQIRRELVM v)
01068 {
01069     SQObject &o = stack_get(v,1);
01070     if (SQ_FAILED(_map_array(_array(o),_array(o),v)))
01071         return SQ_ERROR;
01072     sq_pop(v,1);
01073     return 1;
01074 }
01075
01082 static SQInteger array_reduce(HSQIRRELVM v)
01083 {
01084     SQObject &o = stack_get(v,1);
01085     SQArray *a = _array(o);
01086     SQInteger size = a->Size();
01087     SQObjectPtr res;
01088     SQInteger iterStart;
01089     if (sq_gettop(v)>2) {
01090         res = stack_get(v,3);
01091         iterStart = 0;
01092     } else if (size==0) {
01093         return 0;
01094     } else {
01095         a->Get(0,res);
01096         iterStart = 1;
01097     }
01098     if (size > iterStart) {
01099         SQObjectPtr other;
01100         v->Push(stack_get(v,2));
01101         for (SQInteger n = iterStart; n < size; n++) {
01102             a->Get(n,other);
01103             v->Push(o);
01104             v->Push(res);
01105             v->Push(other);
01106             if (SQ_FAILED(sq_call(v,3,SQTrue,SQFalse))) {
01107                 return SQ_ERROR;
01108             }
01109             res = v->GetUp(-1);
01110             v->Pop();
01111         }
01112         v->Pop();
01113     }
01114     v->Push(res);
01115     return 1;
01116 }
01117
01125 static SQInteger array_filter(HSQIRRELVM v)
01126 {
01127     SQObject &o = stack_get(v,1);
01128     SQArray *a = _array(o);
01129     SQObjectPtr ret = SQArray::Create(_ss(v),0);
01130     SQInteger size = a->Size();
01131     SQObjectPtr val;
01132     for (SQInteger n = 0; n < size; n++) {
01133         a->Get(n,val);
01134         v->Push(o);
01135         v->Push(n);
01136         v->Push(val);
01137         if (SQ_FAILED(sq_call(v,3,SQTrue,SQFalse))) {

```

```

01138         return SQ_ERROR;
01139     }
01140     if(!SQVM::IsFalse(v->GetUp(-1))) {
01141         _array(ret)->Append(val);
01142     }
01143     v->Pop();
01144 }
01145 v->Push(ret);
01146 return 1;
01147 }
01148
01155 static SQInteger array_find(HSQIRRELVM v)
01156 {
01157     SQObject &o = stack_get(v,1);
01158     SQObjectPtr &val = stack_get(v,2);
01159     SQArray *a = _array(o);
01160     SQInteger size = a->Size();
01161     SQObjectPtr temp;
01162     for(SQInteger n = 0; n < size; n++) {
01163         bool res = false;
01164         a->Get(n,temp);
01165         if(SQVM::IsEqual(temp,val,res) && res) {
01166             v->Push(n);
01167             return 1;
01168         }
01169     }
01170     return 0;
01171 }
01172
01187 static bool _sort_compare(HSQIRRELVM v, SQArray *arr, SQObjectPtr &a,SQObjectPtr &b,SQInteger
func,SQInteger &ret)
01188 {
01189     if(func < 0) {
01190         if(!v->ObjCmp(a,b,ret)) return false;
01191     }
01192     else {
01193         SQInteger top = sq_gettop(v);
01194         sq_push(v, func);
01195         sq_pushroottable(v);
01196         v->Push(a);
01197         v->Push(b);
01198         SQObjectPtr *valptr = arr->_values._vals;
01199         SQUnsignedInteger precallsize = arr->_values.size();
01200         if(SQ_FAILED(sq_call(v, 3, SQTrue, SQFalse))) {
01201             if(!sq_isstring( v->_lasterror))
01202                 v->Raise_Error(_SC("compare func failed"));
01203             return false;
01204         }
01205         if(SQ_FAILED(sq_getinteger(v, -1, &ret))) {
01206             v->Raise_Error(_SC("numeric value expected as return value of the compare function"));
01207             return false;
01208         }
01209         if (precallsize != arr->_values.size() || valptr != arr->_values._vals) {
01210             v->Raise_Error(_SC("array resized during sort operation"));
01211             return false;
01212         }
01213         sq_settop(v, top);
01214         return true;
01215     }
01216     return true;
01217 }
01218
01231 static bool _hsort_sift_down(HSQIRRELVM v,SQArray *arr, SQInteger root, SQInteger bottom, SQInteger
func)
01232 {
01233     SQInteger maxChild;
01234     SQInteger done = 0;
01235     SQInteger ret;
01236     SQInteger root2;
01237     while (((root2 = root * 2) <= bottom) && (!done))
01238     {
01239         if (root2 == bottom) {
01240             maxChild = root2;
01241         }
01242         else {
01243             if(!_sort_compare(v,arr,arr->_values[root2],arr->_values[root2 + 1],func,ret))
01244                 return false;
01245             if (ret > 0) {
01246                 maxChild = root2;
01247             }
01248             else {
01249                 maxChild = root2 + 1;
01250             }
01251         }
01252         if(!_sort_compare(v,arr,arr->_values[root],arr->_values[maxChild],func,ret))
01253             return false;
01254     }

```

```

01255         if (ret < 0) {
01256             if (root == maxChild) {
01257                 v->Raise_Error(_SC("inconsistent compare function"));
01258                 return false; // We'd be swapping ourselves. The compare function is incorrect
01259             }
01260
01261             _Swap(arr->_values[root], arr->_values[maxChild]);
01262             root = maxChild;
01263         }
01264         else {
01265             done = 1;
01266         }
01267     }
01268     return true;
01269 }
01270
01284 static bool _hsort(HSQUIRRELVLM v, SQObjectPtr &arr, SQInteger SQ_UNUSED_ARG(1), SQInteger
SQ_UNUSED_ARG(r), SQInteger func)
01285 {
01286     SQArray *a = _array(arr);
01287     SQInteger i;
01288     SQInteger array_size = a->Size();
01289     for (i = (array_size / 2); i >= 0; i--) {
01290         if(!_hsort_sift_down(v, a, i, array_size - 1, func)) return false;
01291     }
01292
01293     for (i = array_size-1; i >= 1; i--)
01294     {
01295         _Swap(a->_values[0], a->_values[i]);
01296         if(!_hsort_sift_down(v, a, 0, i-1, func)) return false;
01297     }
01298     return true;
01299 }
01300
01308 static SQInteger array_sort (HSQUIRRELVLM v)
01309 {
01310     SQInteger func = -1;
01311     SQObjectPtr &o = stack_get(v, 1);
01312     if(_array(o)->Size() > 1) {
01313         if(sq_gettop(v) == 2) func = 2;
01314         if(!_hsort(v, o, 0, _array(o)->Size()-1, func))
01315             return SQ_ERROR;
01316     }
01317     sq_settop(v, 1);
01318     return 1;
01319 }
01320
01328 static SQInteger array_slice(HSQUIRRELVLM v)
01329 {
01330     SQInteger sidx, eidx;
01331     SQObjectPtr o;
01332     if(get_slice_params(v, sidx, eidx, o) == -1) return -1;
01333     SQInteger alen = _array(o)->Size();
01334     if(sidx < 0) sidx = alen + sidx;
01335     if(eidx < 0) eidx = alen + eidx;
01336     if(eidx < sidx) return sq_throwerror(v, _SC("wrong indexes"));
01337     if(eidx > alen || sidx < 0) return sq_throwerror(v, _SC("slice out of range"));
01338     SQArray *arr = SQArray::Create(_ss(v), eidx-sidx);
01339     SQObjectPtr t;
01340     SQInteger count=0;
01341     for(SQInteger i=sidx; i<eidx; i++){
01342         _array(o)->Get(i, t);
01343         arr->Set(count++, t);
01344     }
01345     v->Push(arr);
01346     return 1;
01347 }
01348
01349
01353 const SQRegFunction SQSharedState::_array_default_delegate_funcz[]={
01354     {_SC("len"), default_delegate_len, 1, _SC("a")},
01355     {_SC("append"), array_append, 2, _SC("a")},
01356     {_SC("extend"), array_extend, 2, _SC("aa")},
01357     {_SC("push"), array_append, 2, _SC("a")},
01358     {_SC("pop"), array_pop, 1, _SC("a")},
01359     {_SC("top"), array_top, 1, _SC("a")},
01360     {_SC("insert"), array_insert, 3, _SC("an")},
01361     {_SC("remove"), array_remove, 2, _SC("an")},
01362     {_SC("resize"), array_resize, 2, _SC("an")},
01363     {_SC("reverse"), array_reverse, 1, _SC("a")},
01364     {_SC("sort"), array_sort, -1, _SC("ac")},
01365     {_SC("slice"), array_slice, -1, _SC("ann")},
01366     {_SC("weakref"), obj_delegate_weakref, 1, NULL },
01367     {_SC("tostring"), default_delegate_tostring, 1, _SC(".")},
01368     {_SC("clear"), obj_clear, 1, _SC(".")},
01369     {_SC("map"), array_map, 2, _SC("ac")},

```

```

01370     {_SC("apply"),array_apply,2, _SC("ac")},
01371     {_SC("reduce"),array_reduce,-2, _SC("ac.")},
01372     {_SC("filter"),array_filter,2, _SC("ac")},
01373     {_SC("find"),array_find,2, _SC("a.")},
01374     {NULL, (SQFUNCTION)0,0,NULL}
01375 };
01376
01377 //STRING DEFAULT DELEGATE////////////////////////////////////
01384 static SQInteger string_slice(HSQIRRELVM v)
01385 {
01386     SQInteger sidx,eidx;
01387     SQObjectPtr o;
01388     if(SQ_FAILED(get_slice_params(v,sidx,eidx,o)))return -1;
01389     SQInteger slen = _string(o)->_len;
01390     if(sidx < 0)sidx = slen + sidx;
01391     if(eidx < 0)eidx = slen + eidx;
01392     if(eidx < sidx) return sq_throwerror(v,_SC("wrong indexes"));
01393     if(eidx > slen || sidx < 0) return sq_throwerror(v, _SC("slice out of range"));
01394     v->Push(SQString::Create(_ss(v), &_stringval(o)[sidx],eidx-sidx));
01395     return 1;
01396 }
01397
01404 static SQInteger string_find(HSQIRRELVM v)
01405 {
01406     SQInteger top,start_idx=0;
01407     const SQChar *str,*substr,*ret;
01408     if(((top=sq_gettop(v))>1) && SQ_SUCCEEDED(sq_getstring(v,1,&str)) &&
SQ_SUCCEEDED(sq_getstring(v,2,&substr))) {
01409         if(top>2) sq_getinteger(v,3,&start_idx);
01410         if((sq_getsize(v,1)>start_idx) && (start_idx>=0)) {
01411             ret=scstrchr(&str[start_idx],substr);
01412             if(ret) {
01413                 sq_pushinteger(v, (SQInteger) (ret-str));
01414                 return 1;
01415             }
01416         }
01417         return 0;
01418     }
01419     return sq_throwerror(v,_SC("invalid param"));
01420 }
01421
01427 #define STRING_TOFUNCZ(func) static SQInteger string_##func(HSQIRRELVM v) \
01428 {\
01429     SQInteger sidx,eidx; \
01430     SQObjectPtr str; \
01431     if(SQ_FAILED(get_slice_params(v,sidx,eidx,str)))return -1; \
01432     SQInteger slen = _string(str)->_len; \
01433     if(sidx < 0)sidx = slen + sidx; \
01434     if(eidx < 0)eidx = slen + eidx; \
01435     if(eidx < sidx) return sq_throwerror(v,_SC("wrong indexes")); \
01436     if(eidx > slen || sidx < 0) return sq_throwerror(v,_SC("slice out of range")); \
01437     SQInteger len=_string(str)->_len; \
01438     const SQChar *sthis=_stringval(str); \
01439     SQChar *snew=(_ss(v)->GetScratchPad(sq_rsl(len))); \
01440     memcpy(snew,sthis,sq_rsl(len)); \
01441     for(SQInteger i=sidx;i<eidx;i++) snew[i] = func(sthis[i]); \
01442     v->Push(SQString::Create(_ss(v),snew,len)); \
01443     return 1; \
01444 }
01445
01450 STRING_TOFUNCZ(tolower)
01455 STRING_TOFUNCZ(toupper)
01456
01457
01460 const SQRegFunction SQSharedState::_string_default_delegate_funcz[]={
01461     {_SC("len"),default_delegate_len,1, _SC("s")},
01462     {_SC("tointeger"),default_delegate_tointeger,-1, _SC("sn")},
01463     {_SC("tofloat"),default_delegate_tofloat,1, _SC("s")},
01464     {_SC("tostring"),default_delegate_tostring,1, _SC(".")},
01465     {_SC("slice"),string_slice,-1, _SC("s n n")},
01466     {_SC("find"),string_find,-2, _SC("s s n")},
01467     {_SC("tolower"),string_tolower,-1, _SC("s n n")},
01468     {_SC("toupper"),string_toupper,-1, _SC("s n n")},
01469     {_SC("weakref"),obj_delegate_weakref,1, NULL },
01470     {NULL, (SQFUNCTION)0,0,NULL}
01471 };
01472
01476 const SQRegFunction SQSharedState::_number_default_delegate_funcz[]={
01477     {_SC("tointeger"),default_delegate_tointeger,1, _SC("n|b")},
01478     {_SC("tofloat"),default_delegate_tofloat,1, _SC("n|b")},
01479     {_SC("tostring"),default_delegate_tostring,1, _SC(".")},
01480     {_SC("tochar"),number_delegate_tochar,1, _SC("n|b")},
01481     {_SC("weakref"),obj_delegate_weakref,1, NULL },
01482     {NULL, (SQFUNCTION)0,0,NULL}
01483 };
01484
01485 //CLOSURE DEFAULT DELEGATE////////////////////////////////////

```

```

01492 static SQInteger closure_pcall(HSQIRRELVLM v)
01493 {
01494     return SQ_SUCCEEDED(sq_call(v, sq_gettop(v)-1, SQTrue, SQFalse)) ? 1 : SQ_ERROR;
01495 }
01496
01503 static SQInteger closure_call(HSQIRRELVLM v)
01504 {
01505     SQObjectPtr &c = stack_get(v, -1);
01506     if (sq_type(c) == OT_CLOSURE && (_closure(c)->_function->_bgenerator == false))
01507     {
01508         return sq_tailcall(v, sq_gettop(v) - 1);
01509     }
01510     return SQ_SUCCEEDED(sq_call(v, sq_gettop(v) - 1, SQTrue, SQTrue)) ? 1 : SQ_ERROR;
01511 }
01512
01520 static SQInteger _closure_acall(HSQIRRELVLM v, SQBool raiseerror)
01521 {
01522     SQArray *aparams=_array(stack_get(v, 2));
01523     SQInteger nparams=aparams->Size();
01524     v->Push(stack_get(v, 1));
01525     for(SQInteger i=0; i<nparams; i++) v->Push(aparams->_values[i]);
01526     return SQ_SUCCEEDED(sq_call(v, nparams, SQTrue, raiseerror)) ? 1 : SQ_ERROR;
01527 }
01528
01535 static SQInteger closure_acall(HSQIRRELVLM v)
01536 {
01537     return _closure_acall(v, SQTrue);
01538 }
01539
01546 static SQInteger closure_pacall(HSQIRRELVLM v)
01547 {
01548     return _closure_acall(v, SQFalse);
01549 }
01550
01557 static SQInteger closure_bindenv(HSQIRRELVLM v)
01558 {
01559     if(SQ_FAILED(sq_bindenv(v, 1)))
01560         return SQ_ERROR;
01561     return 1;
01562 }
01563
01570 static SQInteger closure_getroot(HSQIRRELVLM v)
01571 {
01572     if(SQ_FAILED(sq_getclosureroot(v, -1)))
01573         return SQ_ERROR;
01574     return 1;
01575 }
01576
01583 static SQInteger closure_setroot(HSQIRRELVLM v)
01584 {
01585     if(SQ_FAILED(sq_setclosureroot(v, -2)))
01586         return SQ_ERROR;
01587     return 1;
01588 }
01589
01597 static SQInteger closure_getinfos(HSQIRRELVLM v) {
01598     SQObject o = stack_get(v, 1);
01599     SQTable *res = SQTable::Create(_ss(v), 4);
01600     if(sq_type(o) == OT_CLOSURE) {
01601         SQFunctionProto *f = _closure(o)->_function;
01602         SQInteger nparams = f->_nparameters + (f->_varparams ? 1 : 0);
01603         SQObjectPtr params = SQArray::Create(_ss(v), nparams);
01604         SQObjectPtr defparams = SQArray::Create(_ss(v), f->_ndefaultparams);
01605         for(SQInteger n = 0; n<f->_nparameters; n++) {
01606             _array(params)->Set((SQInteger)n, f->_parameters[n]);
01607         }
01608         for(SQInteger j = 0; j<f->_ndefaultparams; j++) {
01609             _array(defparams)->Set((SQInteger)j, _closure(o)->_defaultparams[j]);
01610         }
01611         if(f->_varparams) {
01612             _array(params)->Set(nparams-1, SQString::Create(_ss(v), _SC("..."), -1));
01613         }
01614         res->NewSlot(SQString::Create(_ss(v), _SC("native"), -1), false);
01615         res->NewSlot(SQString::Create(_ss(v), _SC("name"), -1), f->_name);
01616         res->NewSlot(SQString::Create(_ss(v), _SC("src"), -1), f->_sourcename);
01617         res->NewSlot(SQString::Create(_ss(v), _SC("parameters"), -1), params);
01618         res->NewSlot(SQString::Create(_ss(v), _SC("varargs"), -1), f->_varparams);
01619         res->NewSlot(SQString::Create(_ss(v), _SC("defparams"), -1), defparams);
01620     }
01621     else { //OT_NATIVECLOSURE
01622         SQNativeClosure *nc = _nativeclosure(o);
01623         res->NewSlot(SQString::Create(_ss(v), _SC("native"), -1), true);
01624         res->NewSlot(SQString::Create(_ss(v), _SC("name"), -1), nc->_name);
01625         res->NewSlot(SQString::Create(_ss(v), _SC("paramscheck"), -1), nc->_nparamscheck);
01626         SQObjectPtr typecheck;
01627         if(nc->_typecheck.size() > 0) {
01628             typecheck =

```

```

01629         SQArray::Create(_ss(v), nc->_typecheck.size());
01630         for(SQUnsignedInteger n = 0; n<nc->_typecheck.size(); n++) {
01631             _array(typecheck)->Set((SQInteger)n,nc->_typecheck[n]);
01632         }
01633     }
01634     res->NewSlot(SQString::Create(_ss(v),_SC("typecheck"),-1),typecheck);
01635 }
01636 v->Push(res);
01637 return 1;
01638 }
01639
01640 const SQRegFunction SQSharedState::_closure_default_delegate_funcz[]={
01641     {_SC("call"),closure_call,-1,_SC("c")},
01642     {_SC("pcall"),closure_pcall,-1,_SC("c")},
01643     {_SC("acall"),closure_acall,2,_SC("ca")},
01644     {_SC("pacall"),closure_pacall,2,_SC("ca")},
01645     {_SC("weakref"),obj_delegate_weakref,1, NULL },
01646     {_SC("tostring"),default_delegate_tostring,1,_SC(".")},
01647     {_SC("bindenv"),closure_bindenv,2,_SC("c x|y|t")},
01648     {_SC("getinfos"),closure_getinfos,1,_SC("c")},
01649     {_SC("getroot"),closure_getroot,1,_SC("c")},
01650     {_SC("setroot"),closure_setroot,2,_SC("ct")},
01651     {NULL,(SQFUNCTION)0,0,NULL}
01652 };
01653
01654 //GENERATOR DEFAULT DELEGATE
01655 static SQInteger generator_getstatus(HSQIRRELVLM v)
01656 {
01657     SQObject &o=stack_get(v,1);
01658     switch(_generator(o)->_state){
01659         case SQGenerator::eSuspended:v->Push(SQString::Create(_ss(v),_SC("suspended")));break;
01660         case SQGenerator::eRunning:v->Push(SQString::Create(_ss(v),_SC("running")));break;
01661         case SQGenerator::eDead:v->Push(SQString::Create(_ss(v),_SC("dead")));break;
01662     }
01663     return 1;
01664 }
01665
01666 const SQRegFunction SQSharedState::_generator_default_delegate_funcz[]={
01667     {_SC("getstatus"),generator_getstatus,1,_SC("g")},
01668     {_SC("weakref"),obj_delegate_weakref,1, NULL },
01669     {_SC("tostring"),default_delegate_tostring,1,_SC(".")},
01670     {NULL,(SQFUNCTION)0,0,NULL}
01671 };
01672
01673 //THREAD DEFAULT DELEGATE
01674 static SQInteger thread_call(HSQIRRELVLM v)
01675 {
01676     SQObjectPtr o = stack_get(v,1);
01677     if(sq_type(o) == OT_THREAD) {
01678         SQInteger nparams = sq_gettop(v);
01679         _thread(o)->Push(_thread(o)->_roottable);
01680         for(SQInteger i = 2; i<(nparams+1); i++)
01681             sq_move(_thread(o),v,i);
01682         if(SQ_SUCCEEDED(sq_call(_thread(o),nparams,SQTrue,SQTrue))) {
01683             sq_move(v,_thread(o),-1);
01684             sq_pop(_thread(o),1);
01685             return 1;
01686         }
01687         v->_lasterror = _thread(o)->_lasterror;
01688         return SQ_ERROR;
01689     }
01690     return sq_throwerror(v,_SC("wrong parameter"));
01691 }
01692
01693 static SQInteger thread_wakeup(HSQIRRELVLM v)
01694 {
01695     SQObjectPtr o = stack_get(v,1);
01696     if(sq_type(o) == OT_THREAD) {
01697         SQVM *thread = _thread(o);
01698         SQInteger state = sq_getvmstate(thread);
01699         if(state != SQ_VMSTATE_SUSPENDED) {
01700             switch(state) {
01701                 case SQ_VMSTATE_IDLE:
01702                     return sq_throwerror(v,_SC("cannot wakeup a idle thread"));
01703                 case SQ_VMSTATE_RUNNING:
01704                     return sq_throwerror(v,_SC("cannot wakeup a running thread"));
01705                 break;
01706             }
01707         }
01708         SQInteger wakeupret = sq_gettop(v)>1?SQTrue:SQFalse;
01709         if(wakeupret) {
01710             sq_move(thread,v,2);
01711         }
01712         if(SQ_SUCCEEDED(sq_wakeupvm(thread,wakeupret,SQTrue,SQTrue,SQFalse))) {
01713             sq_move(v,thread,-1);
01714         }
01715     }
01716 }

```

```

01740         sq_pop(thread,1); //pop retval
01741         if(sq_getvmstate(thread) == SQ_VMSTATE_IDLE) {
01742             sq_settop(thread,1); //pop roottable
01743         }
01744         return 1;
01745     }
01746     sq_settop(thread,1);
01747     v->_lasterror = thread->_lasterror;
01748     return SQ_ERROR;
01749 }
01750 return sq_throwerror(v,_SC("wrong parameter"));
01751 }
01752
01759 static SQInteger thread_wakeupthrow(HSQUIRRELVm v)
01760 {
01761     SQObjectPtr o = stack_get(v,1);
01762     if(sq_type(o) == OT_THREAD) {
01763         SQVM *thread = _thread(o);
01764         SQInteger state = sq_getvmstate(thread);
01765         if(state != SQ_VMSTATE_SUSPENDED) {
01766             switch(state) {
01767                 case SQ_VMSTATE_IDLE:
01768                     return sq_throwerror(v,_SC("cannot wakeup a idle thread"));
01769                     break;
01770                 case SQ_VMSTATE_RUNNING:
01771                     return sq_throwerror(v,_SC("cannot wakeup a running thread"));
01772                     break;
01773             }
01774         }
01775
01776         sq_move(thread,v,2);
01777         sq_throwobject(thread);
01778         SQBool rethrow_error = SQTrue;
01779         if(sq_gettop(v) > 2) {
01780             sq_getbool(v,3,&rethrow_error);
01781         }
01782         if(SQ_SUCCEEDED(sq_wakeupvm(thread,SQFalse,SQTrue,SQTrue,SQTrue))) {
01783             sq_move(v,thread,-1);
01784             sq_pop(thread,1); //pop retval
01785             if(sq_getvmstate(thread) == SQ_VMSTATE_IDLE) {
01786                 sq_settop(thread,1); //pop roottable
01787             }
01788             return 1;
01789         }
01790         sq_settop(thread,1);
01791         if(rethrow_error) {
01792             v->_lasterror = thread->_lasterror;
01793             return SQ_ERROR;
01794         }
01795         return SQ_OK;
01796     }
01797     return sq_throwerror(v,_SC("wrong parameter"));
01798 }
01799
01806 static SQInteger thread_getstatus(HSQUIRRELVm v)
01807 {
01808     SQObjectPtr &o = stack_get(v,1);
01809     switch(sq_getvmstate(_thread(o))) {
01810         case SQ_VMSTATE_IDLE:
01811             sq_pushstring(v,_SC("idle"),-1);
01812             break;
01813         case SQ_VMSTATE_RUNNING:
01814             sq_pushstring(v,_SC("running"),-1);
01815             break;
01816         case SQ_VMSTATE_SUSPENDED:
01817             sq_pushstring(v,_SC("suspended"),-1);
01818             break;
01819         default:
01820             return sq_throwerror(v,_SC("internal VM error"));
01821     }
01822     return 1;
01823 }
01824
01831 static SQInteger thread_getstackinfos(HSQUIRRELVm v)
01832 {
01833     SQObjectPtr o = stack_get(v,1);
01834     if(sq_type(o) == OT_THREAD) {
01835         SQVM *thread = _thread(o);
01836         SQInteger threadtop = sq_gettop(thread);
01837         SQInteger level;
01838         sq_getinteger(v,-1,&level);
01839         SQRESULT res = __getcallstackinfos(thread,level);
01840         if(SQ_FAILED(res))
01841         {
01842             sq_settop(thread,threadtop);
01843             if(sq_type(thread->_lasterror) == OT_STRING) {
01844                 sq_throwerror(v,_stringval(thread->_lasterror));

```



```

01845         }
01846         else {
01847             sq_throwerror(v,_SC("unknown error"));
01848         }
01849     }
01850     if(res > 0) {
01851         //some result
01852         sq_move(v,thread,-1);
01853         sq_settop(thread,threadtop);
01854         return 1;
01855     }
01856     //no result
01857     sq_settop(thread,threadtop);
01858     return 0;
01859 }
01860 }
01861 return sq_throwerror(v,_SC("wrong parameter"));
01862 }
01863
01864 const SQRegFunction SQSharedState::_thread_default_delegate_funcz[] = {
01865     {_SC("call"), thread_call, -1, _SC("v")},
01866     {_SC("wakeup"), thread_wakeup, -1, _SC("v")},
01867     {_SC("wakeupthrow"), thread_wakeupthrow, -2, _SC("v.b")},
01868     {_SC("getstatus"), thread_getstatus, 1, _SC("v")},
01869     {_SC("weakref"), obj_delegate_weakref, 1, NULL },
01870     {_SC("getstackinfos"), thread_getstackinfos, 2, _SC("vn")},
01871     {_SC("tostring"), default_delegate_tostring, 1, _SC(".")},
01872     {NULL, (SQFUNCTION)0, 0, NULL}
01873 };
01874
01875 static SQInteger class_getattributes(HSQUIRRELV v)
01876 {
01877     return SQ_SUCCEEDED(sq_getattributes(v,-2)) ? 1:SQ_ERROR;
01878 }
01879
01880 static SQInteger class_setattributes(HSQUIRRELV v)
01881 {
01882     return SQ_SUCCEEDED(sq_setattributes(v,-3)) ? 1:SQ_ERROR;
01883 }
01884
01885 static SQInteger class_instance(HSQUIRRELV v)
01886 {
01887     return SQ_SUCCEEDED(sq_createinstance(v,-1)) ? 1:SQ_ERROR;
01888 }
01889
01890 static SQInteger class_getbase(HSQUIRRELV v)
01891 {
01892     return SQ_SUCCEEDED(sq_getbase(v,-1)) ? 1:SQ_ERROR;
01893 }
01894
01895 static SQInteger class_newmember(HSQUIRRELV v)
01896 {
01897     SQInteger top = sq_gettop(v);
01898     SQBool bstatic = SQFalse;
01899     if(top == 5)
01900     {
01901         sq_tobool(v,-1,&bstatic);
01902         sq_pop(v,1);
01903     }
01904     if(top < 4) {
01905         sq_pushnull(v);
01906     }
01907     return SQ_SUCCEEDED(sq_newmember(v,-4,bstatic)) ? 1:SQ_ERROR;
01908 }
01909
01910 static SQInteger class_rawnewmember(HSQUIRRELV v)
01911 {
01912     SQInteger top = sq_gettop(v);
01913     SQBool bstatic = SQFalse;
01914     if(top == 5)
01915     {
01916         sq_tobool(v,-1,&bstatic);
01917         sq_pop(v,1);
01918     }
01919     if(top < 4) {
01920         sq_pushnull(v);
01921     }
01922     return SQ_SUCCEEDED(sq_rawnewmember(v,-4,bstatic)) ? 1:SQ_ERROR;
01923 }
01924
01925 const SQRegFunction SQSharedState::_class_default_delegate_funcz[] = {
01926     {_SC("getattributes"), class_getattributes, 2, _SC("y.")},
01927     {_SC("setattributes"), class_setattributes, 3, _SC("y..")},
01928     {_SC("rawget"), container_rawget, 2, _SC("y")},
01929     {_SC("rawset"), container_rawset, 3, _SC("y")},

```

```

01974     {_SC("rawin"), container_rawexists, 2, _SC("y")},
01975     {_SC("weakref"), obj_delegate_weakref, 1, NULL },
01976     {_SC("tostring"), default_delegate_tostring, 1, _SC(".")},
01977     {_SC("instance"), class_instance, 1, _SC("y")},
01978     {_SC("getbase"), class_getbase, 1, _SC("y")},
01979     {_SC("newmember"), class_newmember, -3, _SC("y")},
01980     {_SC("rawnewmember"), class_rawnewmember, -3, _SC("y")},
01981     {NULL, (SQFUNCTION) 0, 0, NULL}
01982 };
01983
01990 static SQInteger instance_getclass(HSQIRRELVM v)
01991 {
01992     if(SQ_SUCCEEDED(sq_getclass(v, 1)))
01993         return 1;
01994     return SQ_ERROR;
01995 }
01996
02000 const SQRegFunction SQSharedState::_instance_default_delegate_funcz[] = {
02001     {_SC("getclass"), instance_getclass, 1, _SC("x")},
02002     {_SC("rawget"), container_rawget, 2, _SC("x")},
02003     {_SC("rawset"), container_rawset, 3, _SC("x")},
02004     {_SC("rawin"), container_rawexists, 2, _SC("x")},
02005     {_SC("weakref"), obj_delegate_weakref, 1, NULL },
02006     {_SC("tostring"), default_delegate_tostring, 1, _SC(".")},
02007     {NULL, (SQFUNCTION) 0, 0, NULL}
02008 };
02009
02016 static SQInteger weakref_ref(HSQIRRELVM v)
02017 {
02018     if(SQ_FAILED(sq_getweakrefval(v, 1)))
02019         return SQ_ERROR;
02020     return 1;
02021 }
02022
02026 const SQRegFunction SQSharedState::_weakref_default_delegate_funcz[] = {
02027     {_SC("ref"), weakref_ref, 1, _SC("r")},
02028     {_SC("weakref"), obj_delegate_weakref, 1, NULL },
02029     {_SC("tostring"), default_delegate_tostring, 1, _SC(".")},
02030     {NULL, (SQFUNCTION) 0, 0, NULL}
02031 };

```